

```

%%%%%%%%%%
%TA: Raymond Ma
%Group Members: Grace, Yina, Joaquin

%% Intial Parameters for both designs and cases:
n = 1250;                % Number of locations to evaluate bridge failure
L = 1250;                % Length of bridge

x = linspace(1,L,n);    % Define x coordinate

locationA = 1;
locationB = 1060;

%Material properties
SigT = 30;
SigC = 6;
E     = 4000;
TauU  = 4;
TauG  = 2;
mu    = 0.2;

```

```

%% Design Zero: Train
SFD_TrainLoad = zeros(1, n);    % Initialize SFD(x)
BMD_TrainLoad = zeros(1, n);    % Initialize BMD
SFD_TrainLoad1 = zeros(1, n);   % Initialize SFD(x)
BMD_TrainLoad1 = zeros(1, n);   % Initialize BMD
SFD_TrainLoad2 = zeros(1, n);   % Initialize SFD(x)
BMD_TrainLoad2 = zeros(1, n);   % Initialize BMD
SFD_TrainLoad3 = zeros(1, n);   % Initialize SFD(x)
BMD_TrainLoad3 = zeros(1, n);   % Initialize BMD
P = 400/6;

%diaphragms for Design 0
a = zeros(1,length(x));
a(1:550) = 550;
a(551:1060) = 510;
a(1061:1250) = 190;

%Cross Section
%% 2. Define cross-sections
% Design 0
dx = n/L; %length between each x coordinate
xc = [1 dx*L]; % x locations of changes in cross section
bVector = [100 1.27 1.27 10 10 80]; %Base of rectangle segments
hVector = [1.27 72.46 72.46 1.27 1.27 1.27]; %height of rectangle segments
yVector = [74.365 37.5 37.5 73.1 73.1 0.635]; % distances between local centroid of sub

I = zeros(1, length(x)); %Initialize I values
yBar = zeros(1, length(x)); %Initialize Y values
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector); %Calc

cutoffAreaCent = [ 77.46*1.27, 1.27*yBar(1), 1.27*yBar(1)]; %Calculate area from centroid

```

```

yCentCutoff = [ yBar(1)-0.635, yBar(1)/2, yBar(1)/2]; %Calculate y values from centroid

cutoffAreaGlue = [127];
yGlue = [75-yBar(1)-0.635]; % centroid of the cutoff area

Qglue = zeros(1, length(x)); %initialize Q at centroid and Q at glue
Qmax = zeros(1, length(x));
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(1), xc(2), cutoffAreaCent, cutoffAreaGlue,

%Other constants
b = zeros(1,length(x)); %Width of Shear surface
b(:) = 2*1.27;
bGlue = zeros(1,length(x)); %Width of Shear surface of glue
bGlue(:) = 10;
t = zeros(1,length(x)); %Thickness of board (used for plate buckling)
t(:) = 1.27;
height = zeros(1,length(x)); %Height of cross section
height(:) = 75;

%% Train Load: 6 points for the 6 wheels
TLocation = 1; %Train at Start
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation, P, x, SFD_TrainLoad, locationA, loc
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176, P, x, SFD_TrainLoad, location
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164, P, x, SFD_TrainLoad, loc
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176, P, x, SFD_TrainLo
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176 + 164, P, x, SFD_T
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176 + 164 + 176, P, x,

TLocation = 196; %Train at Start
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation, P, x, SFD_TrainLoad2, locationA,
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176, P, x, SFD_TrainLoad2, locat
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164, P, x, SFD_TrainLoad2,
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176, P, x, SFD_Train
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176 + 164, P, x, SFD
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176 + 164 + 176, P,

TLocation = 391; %Train at Start
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation, P, x, SFD_TrainLoad3, locationA,
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176, P, x, SFD_TrainLoad3, locat
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164, P, x, SFD_TrainLoad3,
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176, P, x, SFD_Train
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176 + 164, P, x, SFD
[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176 + 164 + 176, P,

%Find applied stresses
%Shear Stress demand
ShearStress = VStress(I, b, Qmax, SFD_TrainLoad); %Shear Stress at centroid
ShearStressGlue = VStress(I, 20 + 2*1.27, Qglue, SFD_TrainLoad); %Shear Stress at Glue

%Shear Stress Capacity
ShearBuckle = VBuck(I, t, b, height, E, mu, yBar, a, Qmax); %Buckling Shear capacity

%Bending Stress
%Bending Stress Demand

```

```

BendingStressT = MStressT( I,yBar, height, BMD_TrainLoad); %Tension Stress
BendingStressC = MStressC( I,yBar, height, BMD_TrainLoad); %Compression Stress

```

```

%Shear Stress Capacity %Bending Plate Buckling capacity stress

```

```

BendingBuck1 = MBuck( I, t, (100-80)/2, E, mu, 2);
BendingBuck2 = MBuck( I, t, 80-2*1.27, E, mu, 1 );
BendingBuck3 = MBuck( I, t, 75-41.701-1.27, E, mu, 3 );
BendingBuck4 = MBuck( I, t, (100-80)/2, E, mu, 2 );
BendingBuck5 = MBuck( I, t, 75-41.701-1.27, E, mu, 3 );
BendingBuck6 = MBuck( I, t, 41.701-1.27, E, mu, 3 );
BendingBuck7 = MBuck( I, t, 80-2*1.27, E, mu, 1 );
BendingBuck8 = MBuck( I, t, 41.701-1.27, E, mu, 1 );

```

```

%Calculate FOS

```

```

FOSBridge = FOSCalc(ShearStress, ShearStressGlue, BendingStressT, BendingStressC, SigT,

```

```

%Visualization

```

```

%Find max Shear Forces and Bending Moments

```

```

V_Mat = Vfail(I, b, Qmax, TauU); %Max Shear
V_Buck = VfailBuck( I, t, b, height, E, mu, yBar, a, TauU, Qmax); %Max Shear caused by
V_Glue = Vfail(I, bGlue, Qglue, TauU); %Max Shear at glue

```

```

M_MatT = MfailMatT(I, yBar, height, SigT, BMD_TrainLoad); %Max tension

```

```

M_MatC = MfailMatC(I, yBar, height, SigC, BMD_TrainLoad); %Max compression

```

```

M_Buck1 = MfailBuck(I, t, (100-80)/2, E, mu, BMD_TrainLoad, 2, yBar, height, SigC); %Ma
M_Buck2 = MfailBuck(I, t, 80-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck3 = MfailBuck(I, t, 75-41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck4 = MfailBuck(I, t, (100-80)/2, E, mu, BMD_TrainLoad, 2, yBar, height, SigC);
M_Buck5 = MfailBuck(I, t, 75-41.701-1.27, E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck6 = MfailBuck(I, t, 41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck7 = MfailBuck(I, t, 80-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck8 = MfailBuck(I, t, 41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);

```

```

%% Visualization

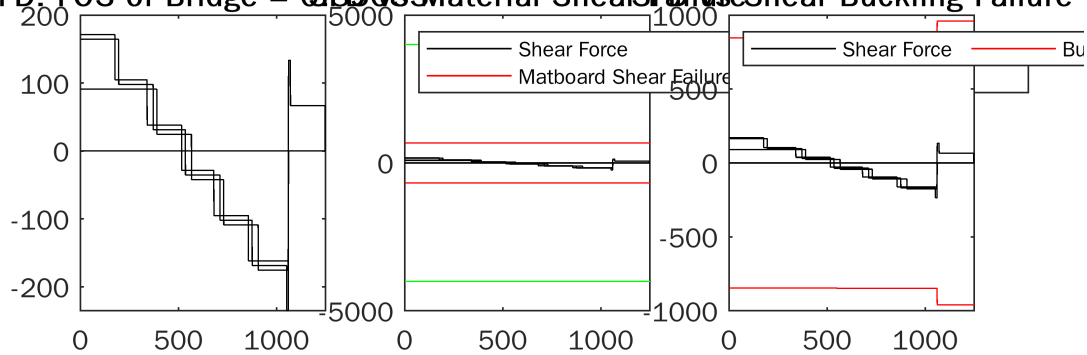
```

```

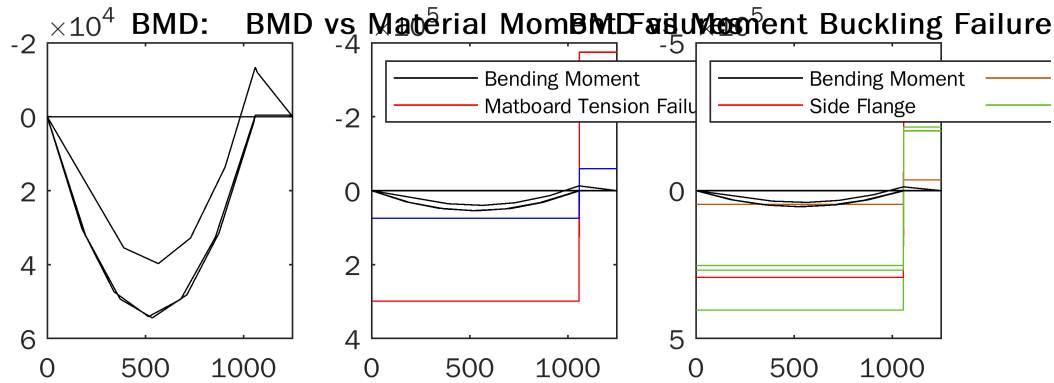
VisualizeTL(x, SFD_TrainLoad, BMD_TrainLoad, SFD_TrainLoad2, BMD_TrainLoad2, SFD_TrainL

```

SFD: FOS of Bridge = 0.5033 Material Shear Failure vs Shear Buckling Failure



BMD: BMD vs Material Moment Buckling Failure



%Design 0 Point Loads

SFD_PL = zeros(1, n); % Initialize SFD(x)

BMD_PL = zeros(1, n);

P = 1; %P = 1 * whatever value, so I can set p to 1

%% 1. Point Loading Analysis (SFD, BMD) Two loads for two points

[SFD_PL, BMD_PL] = ApplyPL(550, P, x, SFD_PL, locationA, locationB); % Construct SFD, BMD

[SFD_PL, BMD_PL] = ApplyPL(L, P, x, SFD_PL, locationA, locationB); % Construct SFD, BMD

%%Calculate Failure Load

Pf = FailLoad(SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2,

%% Visualization

SFD_FAIL = zeros(1, length(x));

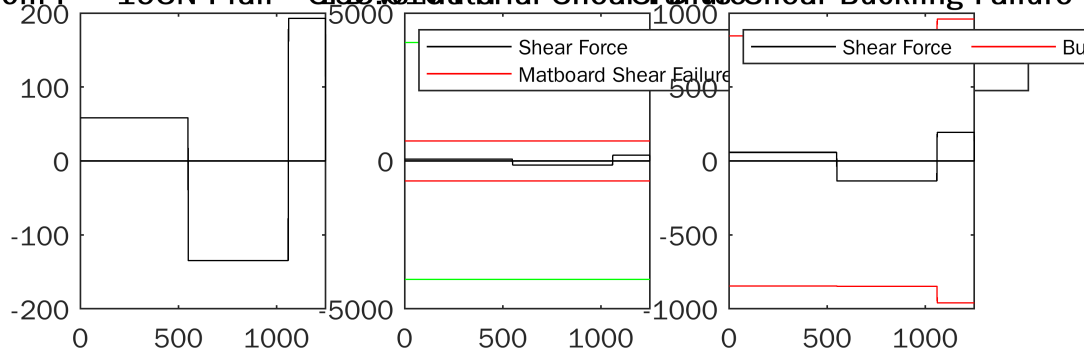
BMD_FAIL = zeros(1, length(x));

[SFD_FAIL, BMD_FAIL] = ApplyPL(550, Pf, x, SFD_FAIL, locationA, locationB); % Construct SFD, BMD

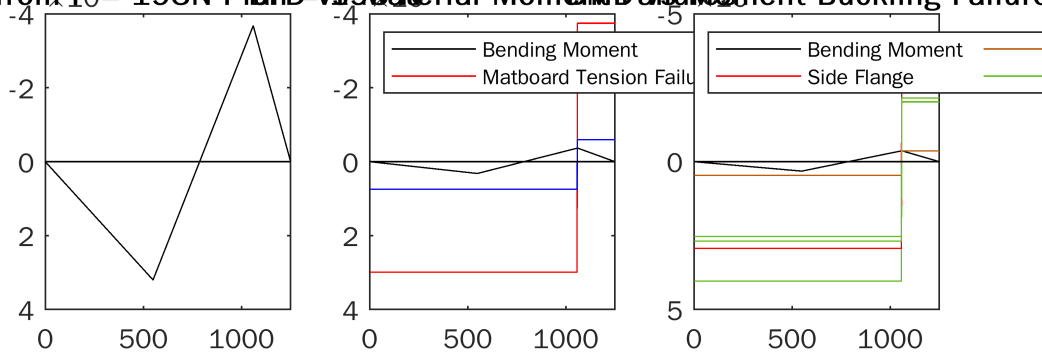
[SFD_FAIL, BMD_FAIL] = ApplyPL(L, Pf, x, SFD_FAIL, locationA, locationB); % Construct SFD, BMD

VisualizePL(x, SFD_FAIL, BMD_FAIL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2,

FD from $P = 193\text{N}$ $P_{fail} = 993.6104\text{N}$ SFD vs SFD Material Shear Failure vs Shear Buckling Failure



BMD from $P = 193\text{N}$ $P_{fail} = 993.6104\text{N}$ BMD vs BMD Material Moment Failure vs Moment Buckling Failure



%Our Design

%Train:

%diaphragms

a = zeros(1,length(x)); %Our design has 3 different diaphragm spacings

a(1:550) = 104;

a(551:1060) = 120;

a(1060:1250) = 53;

%% 2. Define cross-sections

% Cross Section A

dx = n/L;

xc = [1 dx*350 dx*738 dx*838 dx*L]; % x locations of changes in cross section

I = zeros(1, length(x));

yBar = zeros(1, length(x));

Qglue = zeros(1, length(x));

Qmax = zeros(1, length(x));

%Cross Section A

bVector = [100 1.27 1.27 10 10];

hVector = [1.27 120 120 1.27 1.27];

```

yVector = [120.635 60 60 119.365 119.365]; % distances between local centroid of subcom
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector); %This
[I, yBar] = SectionProperties( I, yBar, xc(3)+1, xc(4), bVector, hVector, yVector);
cutoffAreaCent = [ 1.27*yBar(1), 1.27*yBar(1)];
yCentCutoff = [ yBar(1)/2, yBar(1)/2];
cutoffAreaGlue = [127];
yGlue = [121.27-yBar(1)-0.635]; % centroid of the cutoff area
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(1), xc(2), cutoffAreaCent, cutoffAreaGlue,
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(3)+1, xc(4), cutoffAreaCent, cutoffAreaGlue

% Cross Section B
bVector = [100 1.27 1.27 10 10 10 10 70 70];
hVector = [1.27 117.46 117.46 1.27 1.27 1.27 1.27 1.27 1.27];
yVector = [120.635 61.27 61.27 119.365 119.365 3.175 3.175 1.905 0.635]; % distances be
[I, yBar] = SectionProperties( I, yBar, xc(2)+1, xc(3), bVector, hVector, yVector); %TH
[I, yBar] = SectionProperties( I, yBar, xc(4)+1, xc(5), bVector, hVector, yVector);
cutoffAreaCent = [ 127 12.7 12.7 (120-yBar(900))*1.27 (120-yBar(900))*1.27];
yCentCutoff = [ 121.27-yBar(900)-0.635 120-yBar(900)-0.635 120-yBar(900)-0.635 120-yBar
cutoffAreaGlue = [127];
yGlue = [121.27-yBar(1)-0.635]; % centroid of the cutoff area
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(2)+1, xc(3), cutoffAreaCent, cutoffAreaGlue
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(4)+1, xc(5), cutoffAreaCent, cutoffAreaGlue

%Other constants
b = zeros(1,length(x));
b(:) = 2*1.27;
bGlue = zeros(1,length(x));
bGlue(:) = 10;
t = zeros(1,length(x));
t(:) = 1.27;
height = zeros(1,length(x));
height(:) = 120;

% It was unnecessary to calculate the SFD and BMD again since it was already
% done for Design 0

%Find applied stresses
%Shear Stress demand
ShearStress = VStress(I, b, Qmax, SFD_TrainLoad);
ShearStressGlue = VStress(I, 20+ 2*1.27, Qglue, SFD_TrainLoad);

%Shear Stress Capacity
ShearBuckle = VBuck(I, t, b, height, E, mu, yBar, a, Qmax);

%Bending Stress
%Bending Stress Demand
BendingStressT = MStressT( I,yBar, height, BMD_TrainLoad);
BendingStressC = MStressC( I,yBar, height, BMD_TrainLoad);

%Shear Stress Capacity
%Cross Section A
BendingBuck1 = MBuck( I, t, 15, E, mu, 2);
BendingBuck2 = MBuck( I, t, 70-2*(1.27), E, mu, 1);
BendingBuck3 = MBuck( I, t, 39.859, E, mu, 3 );

```

```

BendingBuck4 = MBuck( I, t, 80.1411, E, mu, 3 );

%Cross Section B
BendingBuck5 = MBuck( I, t, 56.4858-2*1.27, E, mu, 3 );
BendingBuck6 = MBuck( I, t, 117.46-56.4858, E, mu, 3 );
BendingBuck7 = MBuck( I, 2*t, 70-2*1.27, E, mu, 1 );
BendingBuck8 = MBuck( I, t, 56.4858-2*1.27, E, mu, 3 );

FOSBridge = FOSCalc(ShearStress, ShearStressGlue, BendingStressT, BendingStressC, SigT,

%Visualization

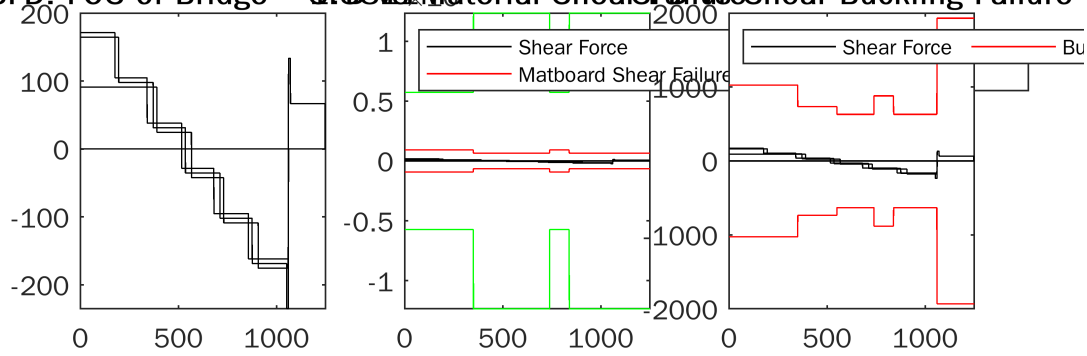
V_Mat = Vfail(I, b, Qmax, TauU);
V_Buck = VfailBuck( I, t, b, height, E, mu, yBar, a, TauU, Qmax);
V_Glue = Vfail(I, bGlue, Qglue, TauU);

M_MatT = MfailMatT(I, yBar, height, SigT, BMD_TrainLoad);
M_MatC = MfailMatC(I, yBar, height, SigC, BMD_TrainLoad);
M_Buck1 = MfailBuck(I, t, 15, E, mu, BMD_TrainLoad, 2, yBar, height, SigC);
M_Buck2 = MfailBuck(I, t, 70-2*(1.27), E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck3 = MfailBuck(I, t, 39.859, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck4 = MfailBuck(I, t, 80.1411, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck5 = MfailBuck(I, t, 56.4858-2*1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck6 = MfailBuck(I, t, 117.46-56.4858, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck7 = MfailBuck(I, 2*t, 70-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck8 = MfailBuck(I, t, 56.4858-2*1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);

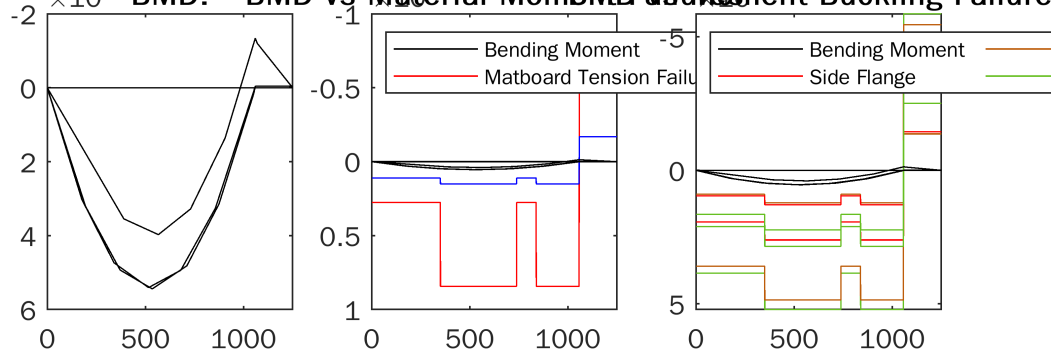
%% Visualization
VisualizeTL(x, SFD_TrainLoad, BMD_TrainLoad, SFD_TrainLoad2, BMD_TrainLoad2, SFD_TrainLoad2, BMD_TrainLoad2);

```

SFD: FOS of Bridge = 3.8814 Material Shear Failure vs Shear Buckling Failure



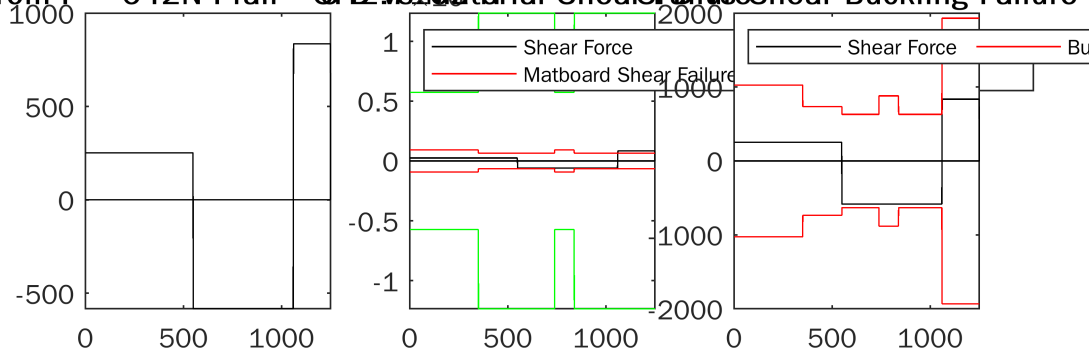
BMD: BMD vs Material Moment Buckling Failure



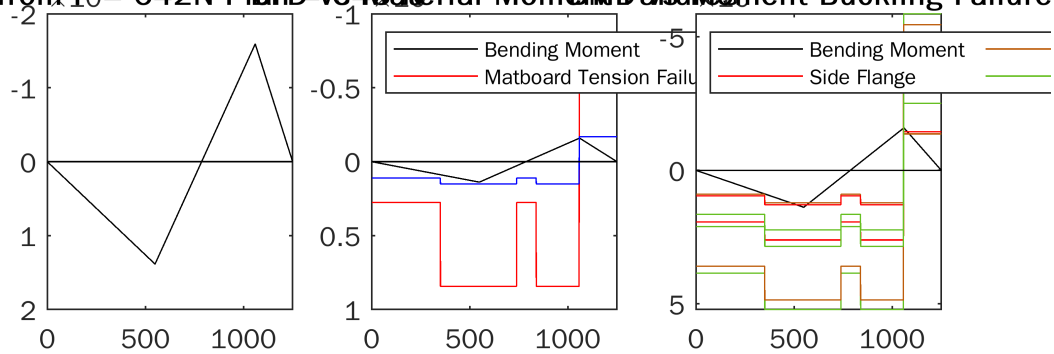
%Point Load our Design:

```
Pf = FailLoad( SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2,
[SFD_FAIL, BMD_FAIL] = ApplyPL(550, Pf, x, SFD_FAIL, locationA, locationB); % Construct
[SFD_FAIL, BMD_FAIL] = ApplyPL(L, Pf, x, SFD_FAIL, locationA, locationB); % Construct
VisualizePL(x, SFD_FAIL, BMD_FAIL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Bu
```

FD from P = 642N Pfail = 542.7102N Material Shear Failure vs Shear Buckling Failure



BMD from P = 642N Pfail = 642.6436N Material Moment Failure vs Moment Buckling Failure



```
% Train failure Functions:
```

```
function [ ShearFail ] = VStress(I, b, Qcent, SFD )
%Calculates the Shear Stress at Applied load for every x value

    ShearFail = ((SFD .* Qcent) ./ (I .* b));
end

function [ ShearBuckle ] = VBuck( I, t, b, height, E, mu, y, a, Qcent )
% Calculates the Shear Buckle capacity at every x value

    %pi
    pi = 355/113; %Pi approximation

    for i = 1:length(I)
        ShearBuckle(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 + (t(i)/a(i)))
    end

end

function [ BendingTensionStress ] = MStressT( I,y, height, BMD ) %Tension
% Calculates Tension Stress at every x value based on applied load
```

```

BendingTensionStress = zeros(1,length(I));

for i = 1 : length(I)
    ybot = y(i);
    ytop = height(i) - y(i);

    if BMD(i) >= 0 % If the moment is positive, the tension failure will be at the
        BendingTensionStress(i) = (BMD(i) * ybot) / I(i);
    elseif BMD < 0 % If the moment is negative, the tension failure will be at the
        BendingTensionStress(i) = (BMD(i) * ytop) / I(i);
    end
end

end

function [ BendingCompressionStress ] = MStressC( I,y, height, BMD ) %Compression
% % Calculates Compression Stress at every x value based on applied load

BendingCompressionStress = zeros(1,length(I));

for i = 1:length(I)

    ybot = y(i);
    ytop = height(i) - y(i);

    if BMD(i) >= 0 % If the moment is positive, the compression failure will be at
        BendingCompressionStress(i) = (BMD(i) * ytop) / I(i);
    elseif BMD(i) > 0 % If the moment is negative, the compression failure will be
        BendingCompressionStress(i) = (BMD(i) * ybot) / I(i);
    end
end

end

function [ MBuckStress ] = MBuck( I, t, bBuck, E, mu, caseNum )
% Calculates Bending Capacity based on plate buckling for every x value

    if caseNum == 1
        k = 4;
    end
    if caseNum == 2
        k = 0.425;
    end
    if caseNum == 3
        k = 6;
    end

    for i = 1:length(I)
        MBuckStress(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;
    end

end

```

```

function [ FOS ] = FOSCalc( ShearFail, GlueFail, BendingTensionStress, BendingCompressionStress,
    %Finds FOS based on every capacity and every demand -> FOS =
    %Capacity/Demand
    ShearFOS = min(TauCapacity ./ abs(ShearFail));
    GlueFOS = min(GlueCapacity ./ abs(GlueFail));
    ShearBucklingFOS = min(TauBuckling ./ abs(ShearFail));
    TensionFOS = min(TensionCapacity ./ abs(BendingTensionStress));
    CompressionFOS = min(CompressionCapacity ./ abs(BendingCompressionStress));
    CompressionBucklingFOS1 = min(CompressionBucklingCapacity1 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS2 = min(CompressionBucklingCapacity2 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS3 = min(CompressionBucklingCapacity3 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS4 = min(CompressionBucklingCapacity4 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS5 = min(CompressionBucklingCapacity5 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS6 = min(CompressionBucklingCapacity6 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS7 = min(CompressionBucklingCapacity7 ./ abs(BendingCompressionStress));
    CompressionBucklingFOS8 = min(CompressionBucklingCapacity8 ./ abs(BendingCompressionStress));

    %Finds the minimum of the FOS
    FOSes = [ShearFOS GlueFOS ShearBucklingFOS TensionFOS CompressionFOS CompressionBucklingFOS1
        CompressionBucklingFOS2 CompressionBucklingFOS3 CompressionBucklingFOS4
        CompressionBucklingFOS5 CompressionBucklingFOS6 CompressionBucklingFOS7
        CompressionBucklingFOS8];
    FOS = min(FOSes);
end

%Cross Section Functions
function [I, yBar] = SectionProperties( I, yBar, xStart, xEnd, bVector, hVector, yVector)
    % Input: Geometric Inputs. Format will depend on user
    % Output: Sectional Properties at every value of x. Each property is a 1-D array of

    % finding global centroid
    for i = xStart : xEnd
        sumAY = 0;
        sumA = 0;

        for j = 1:length(bVector)
            sumAY = sumAY + bVector(j) * hVector(j) * yVector(j);
            sumA = sumA + bVector(j) * hVector(j);
        end

        yBar(i) = sumAY/sumA;

        %finding I
        for j = 1:length(bVector)
            I(i) = I(i) + (bVector(j)*hVector(j)^3)/12 + bVector(j)*hVector(j)*(yVector(j)-yBar(i))^2;
        end
    end

end

end

function [Qmax, Qglue] = calculateQ( Qmax, Qglue, xStart, xEnd, cutoffAreaCent, cutoffArea)
    for i = xStart : xEnd
        %finding Q at centroid
    end
end

```

```

    for j = 1:length(cutoffAreaCent)
        Qmax(i) = Qmax(i) + cutoffAreaCent(j)*yCentCutoff(j);
    end

    %finding Q at glue
    for j = 1:length(cutoffAreaGlue)
        Qglue(i) = Qglue(i) + cutoffAreaGlue(j)*yGlueCutoff(j);
    end

end

end

function [] = VisualizeTL(x, SFD_PL, BMD_PL, SFD2, BMD2, SFD3, BMD3, V_Mat, V_Glue, V_E
    %Plots all outputs of design process
    figure;

    axis = zeros(1,length(x));
    hold on;
    %shear force diagram
    subplot(2, 3, 1)
    plot(x, SFD_PL, "k")
    hold on
    plot(x, SFD2, "k")
    hold on
    plot(x, SFD3, "k")
    hold on
    plot(x,axis,"k")
    hold off
    title1 = strcat("SFD: FOS of Bridge = ", num2str(FOS));
    title(title1)

    %shear force vs mat shear fail
    subplot(2, 3, 2)
    plot(x, SFD_PL, "k")
    hold on
    plot(x, V_Mat, "r")
    hold on
    plot(x, V_Glue, "g")
    hold on
    plot(x, -V_Mat, "r")
    hold on
    plot(x, -V_Glue, "g")
    hold on
    plot(x, SFD2, "k")
    hold on
    plot(x, SFD3, "k")
    hold on
    hold on
    plot(x,axis,"k")
    hold off
    legend("Shear Force", "Matboard Shear Failure", "Glue Shear Failure", 'Location', 'none')
    title("SFD vs Material Shear Failure")

```

```

subplot(2, 3, 3)
plot(x, SFD_PL, "k")
hold on
plot(x, V_Buck, "r")
hold on
plot(x, -V_Buck, "r")
hold on
plot(x, SFD2, "k")
hold on
plot(x, SFD3, "k")
hold on
plot(x,axis,"k")
hold off
legend("Shear Force", "Buckling Failure", 'Location','northwest','NumColumns',2)
title("SFD vs Shear Buckling Failure")

%BMD Diagrams
subplot(2, 3, 4)
plot(x, BMD_PL, "k")
hold on
plot(x, BMD2, "k")
hold on
plot(x, BMD3, "k")
hold on
plot(x,axis,"k")
hold on
title2 = strcat("BMD:");
title(title2)
set(gca, 'YDir', 'reverse')
set(gca, 'YDir', 'reverse')
hold off

subplot(2, 3, 5)
plot(x, BMD_PL, "k")
hold on
plot(x, M_MatT, "r")
hold on
plot(x, M_MatC, "b")
hold on
plot(x, BMD2, "k")
hold on
plot(x, BMD3, "k")
hold on
plot(x,axis,"k")
hold on
legend("Bending Moment", "Matboard Tension Failure", "Matboard Compression Failure",
title("BMD vs Material Moment Failures")
set(gca, 'YDir', 'reverse')
hold off

subplot(2, 3, 6)
plot(x, BMD_PL, "k")
hold on
plot(x, M_Buck1, "-r")

```

```

hold on
plot(x, M_Buck2, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck3, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck4, "-r")
hold on
plot(x, M_Buck5, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck6, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck7, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck8, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, BMD2, "k")
hold on
plot(x, BMD3, "k")
hold on
plot(x,axis,"k")
hold on
%legend("Bending Moment", "Mid Flange Buckling", "Side Flange Buckling", "Web Compr
title("BMD vs Moment Buckling Failures")
legend("Bending Moment", "Side Flange", "Mid Flange", "Web Flange", 'Location', 'north
set(gca, 'YDir', 'reverse')
hold off

```

end

```

%%Point Load Functions:
function [ SFD, BMD ] = ApplyPL( xP, P, x, SFD, locationA, locationB ) %(Location, Force)
% Constructs SFD and BMD for every x value
% Can be iterated since SFD and BMD diagrams can be added together

%Calculate reaction forces
SupportB = ((xP * P) / (locationB)); %Using Sum of Moments = 0
SupportA = (P - SupportB); %Using sum of vertical force = 0

%Fill out SFD diagram as if by hand
SFDLocal = zeros(1, length(SFD));
if xP < locationB %Force is closer to the left side than support B
    for i = 1:(xP-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = xP:locationB %SFD after new load
        SFDLocal(i) = (SupportA - P);
    end

    for i = 1:length(SFD) %add the new SFD with the old ones

```

```

        SFD(i) = SFD(i) + SFDLocal(i);
    end

else %Force is farther from left side than support B
    for i = 1:(locationB-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = locationB:xP-1 %SFD after new load
        SFDLocal(i) = (SupportA + SupportB);
    end

    for i = xP:length(SFD)
        SFDLocal(i) = 0;
    end

    for i = 1:length(SFD) %add the new SFD with the old ones
        SFD(i) = SFD(i) + SFDLocal(i);
    end
end

%form BMD:
dx = x(2) - x(1);
BMD = cumsum((SFD * dx));
end

function [ V_fail ] = Vfail(I, b, Qcent, TauU )
% Calculates Shear Fail based on Tau Ultimate

V_fail = TauU .* I .* b ./ Qcent;
end

function [ V_Buck ] = VfailBuck( I, t, b, height, E, mu, y, a, TauU, Qcent )
% Calculates Shear Fail based on Buckling

%pi
pi = 355/113;

for i = 1:length(I)
    sigma(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 + (t(i)/a(i))^2);
end

%Calculate buckle fail by using the above formula
V_Buck = Vfail(I, b, Qcent, sigma);

end

function [ M_MatT ] = MfailMatT( I,y, height, SigT, BMD )
% Calculates Tension Fail

for i = 1 : length(I)

    ybot = y;
    ytop = height(i) - y; %I need somesort of height vector

```

```

        if BMD(i) > 0 % If the moment is positive, the tension failure will be at the b
            M_MatT(i) = (SigT * I(i)) / ybot(i);
        elseif BMD(i) < 0 % If the moment is negative, the tension failure will be at t
            M_MatT(i) = (-SigT * I(i)) / ytop(i);
        end
    end
end

function [ M_MatC ] = MfailMatC( I,y, height, SigC, BMD )
% Calculates Compression Fail

ybot = y;
ytop = height - y;

for i = 1 : length(I)
    if BMD(i) > 0 % If the moment is positive, the compression failure will be at t
        M_MatC(i) = (SigC * I(i)) / ytop(i);
    elseif BMD(i) < 0 % If the moment is negative, the compression failure will be
        M_MatC(i) = (-SigC * I(i)) / ybot(i);
    end
end
end

function [ M_Buck ] = MfailBuck( I, t, bBuck, E, mu, BMD, caseNum, y, height, SigC )
% Calculates Bending Fail based on buckling

%Assign a k value based on the case
if caseNum == 1
    k = 4;
end
if caseNum == 2
    k = 0.425;
end
if caseNum == 3
    k = 6;
end

for i = 1:length(I)
    sigma(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;
end

%Calculate buckle fail using the above function
M_Buck = MfailMatC(I,y, height, min(sigma), BMD);

end

function [ Pf ] = FailLoad( SFD, BMD, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_buck1, M
%Calculate the point load that causes the failure

%Flexural Stress
%Because P = 1, the BMD and SFD are simply the coefficients of P. By
%dividing them, we are finding P despite the fact that there is no actual P
%variable.

```

```

CompressionFailure = abs(M_MatC) ./ abs(BMD);
TensionFailure = abs(M_MatT) ./ abs(BMD);
BendingBucklingFailure1 = abs(M_buck1) ./ abs(BMD);
BendingBucklingFailure2 = abs(M_buck2) ./ abs(BMD);
BendingBucklingFailure3 = abs(M_buck3) ./ abs(BMD);
BendingBucklingFailure4 = abs(M_buck4) ./ abs(BMD);
BendingBucklingFailure5 = abs(M_buck5) ./ abs(BMD);
BendingBucklingFailure6 = abs(M_buck6) ./ abs(BMD);
BendingBucklingFailure7 = abs(M_buck7) ./ abs(BMD);
BendingBucklingFailure8 = abs(M_buck8) ./ abs(BMD);

%Shear Stress
ShearFailureMat = V_Mat ./ abs(SFD);
ShearFailureGlue = V_Glue ./ abs(SFD);
ShearBucklingFailure = V_Buck ./ abs(SFD);

%Find the lowest Pf
Pforces = [min(CompressionFailure) min(TensionFailure) min(BendingBucklingFailure1) min
Pf = min(Pforces);

end

function [] = VisualizePL(x, SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_E

    axis = zeros(1,length(x));
    %Plots all outputs of design process
    figure;

    hold on;
    %shear force diagram
    subplot(2, 3, 1)
    plot(x, SFD_PL, "k")
    hold on
    plot(x,axis,"k")
    hold off
    title1 = strcat("SFD from P = ", int2str(floor(Pf)), "N Pfail = ", num2str(Pf), " N")
    title(title1)

    %shear force vs mat shear fail
    subplot(2, 3, 2)
    plot(x, SFD_PL, "k")
    hold on
    plot(x, V_Mat, "r")
    hold on
    plot(x, V_Glue, "g")
    hold on
    plot(x, -V_Mat, "r")
    hold on
    plot(x, -V_Glue, "g")
    hold on
    hold on
    plot(x,axis,"k")
    legend("Shear Force", "Matboard Shear Failure", "Glue Shear Failure", 'Location', 'non

```

```

title("SFD vs Material Shear Failure")
hold off

subplot(2, 3, 3)
plot(x, SFD_PL, "k")
hold on
plot(x, V_Buck, "r")
hold on
plot(x, -V_Buck, "r")
hold on
hold on
plot(x,axis,"k")
legend("Shear Force", "Buckling Failure", 'Location','northwest','NumColumns',2)
title("SFD vs Shear Buckling Failure")
hold off

%BMD Diagrams
subplot(2, 3, 4)
plot(x, BMD_PL, "k")
hold on
hold on
plot(x,axis,"k")
title2 = strcat("BMD from P = ", int2str(floor(Pf)), "N Pfail = ", int2str(Pf), " N")
title(title2)
set(gca, 'YDir', 'reverse')
set(gca, 'YDir', 'reverse')
hold off

subplot(2, 3, 5)
plot(x, BMD_PL, "k")
hold on
plot(x, M_MatT, "r")
hold on
plot(x, M_MatC, "b")
hold on
plot(x,axis,"k")
hold on
legend("Bending Moment", "Matboard Tension Failure", "Matboard Compression Failure",
title("BFD vs Material Moment Failures")
set(gca, 'YDir', 'reverse')
hold off

subplot(2, 3, 6)
plot(x, BMD_PL, "-k")
hold on
plot(x, M_Buck1, "-r")
hold on
plot(x, M_Buck2, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck3, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck4, "-r")
hold on
plot(x, M_Buck5, 'Color', [0.4, 0.75, 0.13])

```

```

hold on
plot(x, M_Buck6, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck7, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck8, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x,axis,"k")
hold on
%legend("Bending Moment", "Mid Flange Buckling", "Side Flange Buckling", "Web Compr
title("BMD vs Moment Buckling Failures")
legend("Bending Moment", "Side Flange", "Mid Flange", "Web Flange", 'Location','north
set(gca, 'YDir', 'reverse')
hold off

```

```

end

```

```
%%%%%%%%%
```

```
%TA: Raymond Ma
```

```
%Group Members: Grace, Yina, Joaquin
```

```
Note: This exists because the matlab pdf got cut off
```

```
%%% Intial Parameters for both designs and cases:
```

```
n = 1250;                % Number of locations to evaluate bridge failure
```

```
L = 1250;                % Length of bridge
```

```
x = linspace(1,L,n);    % Define x coordinate
```

```
locationA = 1;
```

```
locationB = 1060;
```

```
%Material properties
```

```
SigT = 30;
```

```
SigC = 6;
```

```
E     = 4000;
```

```
TauU = 4;
```

```
TauG = 2;
```

```
mu    = 0.2;
```

```
%%% Design Zero: Train
```

```
SFD_TrainLoad = zeros(1, n);    % Initialize SFD(x)
```

```
BMD_TrainLoad = zeros(1, n);    % Initialize BMD
```

```
SFD_TrainLoad1 = zeros(1, n);   % Initialize SFD(x)
```

```
BMD_TrainLoad1 = zeros(1, n);   % Initialize BMD
```

```
SFD_TrainLoad2 = zeros(1, n);   % Initialize SFD(x)
```

```
BMD_TrainLoad2 = zeros(1, n);   % Initialize BMD
```

```

SFD_TrainLoad3 = zeros(1, n);      % Initialize SFD(x)
BMD_TrainLoad3 = zeros(1, n);      % Initialize BMD
P = 400/6;

```

```

%diaphragms for Design 0

```

```

a = zeros(1,length(x));
a(1:550) = 550;
a(551:1060) = 510;
a(1061:1250) = 190;

```

```

%Cross Section

```

```

%% 2. Define cross-sections

```

```

% Design 0

```

```

dx = n/L; %length between each x coordinate
xc = [1 dx*L]; % x locations of changes in cross section
bVector = [100 1.27 1.27 10 10 80]; %Base of rectangle segments
hVector = [1.27 72.46 72.46 1.27 1.27 1.27]; %height of rectangle segments
yVector = [74.365 37.5 37.5 73.1 73.1 0.635]; % distances between local
centroid of subcomponents of sections from bottom
I = zeros(1, length(x)); %Initialize I values
yBar = zeros(1, length(x)); %Initialize Y values
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector,
yVector); %Calculate I and Y values at every instance
cutoffAreaCent = [ 77.46*1.27, 1.27*yBar(1), 1.27*yBar(1)]; %Calculate area
from centroid for Q calc
yCentCutoff = [ yBar(1)-0.635, yBar(1)/2, yBar(1)/2]; %Calculate y values from
centroid for Q calc
cutoffAreaGlue = [127];
yGlue = [75-yBar(1)-0.635]; % centroid of the cutoff area
Qglue = zeros(1, length(x)); %initialize Q at centriod and Q at glue
Qmax = zeros(1, length(x));

```

```
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(1), xc(2), cutoffAreaCent, cutoffAreaGlue, yCentCutoff, yGlue); %Calculate Q at centroid and Q at glue
```

```
%Other constants
```

```
b = zeros(1,length(x)); %Width of Shear surface
```

```
b(:) = 2*1.27;
```

```
bGlue = zeros(1,length(x)); %Width of Shear surface of glue
```

```
bGlue(:) = 10;
```

```
t = zeros(1,length(x)); %Thickness of board (used for plate buckling)
```

```
t(:) = 1.27;
```

```
height = zeros(1,length(x)); %Height of cross section
```

```
height(:) = 75;
```

```
%% Train Load: 6 points for the 6 wheels
```

```
TLocation = 1; %Train at Start
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation, P, x, SFD_TrainLoad, locationA, locationB);
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176, P, x, SFD_TrainLoad, locationA, locationB);
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164, P, x, SFD_TrainLoad, locationA, locationB);
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176, P, x, SFD_TrainLoad, locationA, locationB);
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176 + 164, P, x, SFD_TrainLoad, locationA, locationB);
```

```
[SFD_TrainLoad, BMD_TrainLoad] = ApplyPL(TLocation + 176 + 164 + 176 + 164 + 176, P, x, SFD_TrainLoad, locationA, locationB);
```

```
TLocation = 196; %Train at Start
```

```
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation, P, x, SFD_TrainLoad2, locationA, locationB);
```

```
[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176, P, x,
SFD_TrainLoad2, locationA, locationB);

[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164, P, x,
SFD_TrainLoad2, locationA, locationB);

[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176, P, x,
SFD_TrainLoad2, locationA, locationB);

[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176 + 164,
P, x, SFD_TrainLoad2, locationA, locationB);

[SFD_TrainLoad2, BMD_TrainLoad2] = ApplyPL(TLocation + 176 + 164 + 176 + 164 +
176, P, x, SFD_TrainLoad2, locationA, locationB);
```

```
TLocation = 391; %Train at Start

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation, P, x, SFD_TrainLoad3,
locationA, locationB);

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176, P, x,
SFD_TrainLoad3, locationA, locationB);

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164, P, x,
SFD_TrainLoad3, locationA, locationB);

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176, P, x,
SFD_TrainLoad3, locationA, locationB);

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176 + 164,
P, x, SFD_TrainLoad3, locationA, locationB);

[SFD_TrainLoad3, BMD_TrainLoad3] = ApplyPL(TLocation + 176 + 164 + 176 + 164 +
176, P, x, SFD_TrainLoad3, locationA, locationB);
```

```
%Find applied stresses
```

```
%Shear Stress demand
```

```
ShearStress = VStress(I, b, Qmax, SFD_TrainLoad); %Shear Stress at centroid
```

```
ShearStressGlue = VStress(I, 20 + 2*1.27, Qglue, SFD_TrainLoad); %Shear Stress
at Glue
```

```
%Shear Stress Capacity
```

```
ShearBuckle = VBuck(I, t, b, height, E, mu, yBar, a, Qmax); %Buckling Shear
capacity
```

```
%Bending Stress
```

```
%Bending Stress Demand
```

```
BendingStressT = MStressT( I,yBar, height, BMD_TrainLoad); %Tension Stress
```

```
BendingStressC = MStressC( I,yBar, height, BMD_TrainLoad); %Compression Stress
```

```
%Shear Stress Capacity %Bending Plate Buckling capacity stress
```

```
BendingBuck1 = MBuck( I, t, (100-80)/2, E, mu, 2);
```

```
BendingBuck2 = MBuck( I, t, 80-2*1.27, E, mu, 1 );
```

```
BendingBuck3 = MBuck( I, t, 75-41.701-1.27, E, mu, 3 );
```

```
BendingBuck4 = MBuck( I, t, (100-80)/2, E, mu, 2 );
```

```
BendingBuck5 = MBuck( I, t, 75-41.701-1.27, E, mu, 3 );
```

```
BendingBuck6 = MBuck( I, t, 41.701-1.27, E, mu, 3 );
```

```
BendingBuck7 = MBuck( I, t, 80-2*1.27, E, mu, 1 );
```

```
BendingBuck8 = MBuck( I, t, 41.701-1.27, E, mu, 1 );
```

```
%Calculate FOS
```

```
FOSBridge = FOSCalc(ShearStress, ShearStressGlue, BendingStressT,  
BendingStressC, SigT, SigC, BendingBuck1, BendingBuck2, BendingBuck3,  
BendingBuck4, BendingBuck5, BendingBuck6, BendingBuck7, BendingBuck8, TauU,  
TauG, ShearBuckle);
```

```
%Visualization
```

```
%Find max Shear Forces and Bending Moments
```

```
V_Mat = Vfail(I, b, Qmax, TauU); %Max Shear
```

```
V_Buck = VfailBuck( I, t, b, height, E, mu, yBar, a, TauU, Qmax); %Max Shear  
caused by buckling
```

```
V_Glue = Vfail(I, bGlue, Qglue, TauU); %Max Shear at glue
```

```
M_MatT = MfailMatT(I, yBar, height, SigT, BMD_TrainLoad); %Max tension
```

```
M_MatC = MfailMatC(I, yBar, height, SigC, BMD_TrainLoad); %Max compression
```

```

M_Buck1 = MfailBuck(I, t, (100-80)/2, E, mu, BMD_TrainLoad, 2, yBar, height,
SigC); %Max compression caused by buckling

M_Buck2 = MfailBuck(I, t, 80-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height,
SigC);

M_Buck3 = MfailBuck(I, t, 75-41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar,
height, SigC);

M_Buck4 = MfailBuck(I, t, (100-80)/2, E, mu, BMD_TrainLoad, 2, yBar, height,
SigC);

M_Buck5 = MfailBuck(I, t, 75-41.701-1.27, E, mu, BMD_TrainLoad, 1, yBar,
height, SigC);

M_Buck6 = MfailBuck(I, t, 41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar, height,
SigC);

M_Buck7 = MfailBuck(I, t, 80-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height,
SigC);

M_Buck8 = MfailBuck(I, t, 41.701-1.27, E, mu, BMD_TrainLoad, 3, yBar, height,
SigC);

%% Visualization

VisualizeTL(x, SFD_TrainLoad, BMD_TrainLoad, SFD_TrainLoad2, BMD_TrainLoad2,
SFD_TrainLoad3, BMD_TrainLoad3, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1,
M_Buck2, M_Buck3,M_Buck4,M_Buck5,M_Buck6, M_Buck7, M_Buck8, FOSBridge);

```

```

%Design 0 Point Loads

```

```

SFD_PL = zeros(1, n);      % Initialize SFD(x)

```

```

BMD_PL = zeros(1, n);

```

```

P = 1; %P = 1 * whatever value, so I can set p to 1

```

```

%% 1. Point Loading Analysis (SFD, BMD) Two loads for two points

```

```

[SFD_PL, BMD_PL] = ApplyPL(550, P, x, SFD_PL, locationA, locationB); %
Construct SFD, BMD

```

```

[SFD_PL, BMD_PL] = ApplyPL(L, P, x, SFD_PL, locationA, locationB); %
Construct SFD, BMD

```

```
%%Calculate Failure Load
```

```
Pf = FailLoad( SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1,  
M_Buck2, M_Buck3, M_Buck4, M_Buck5, M_Buck6, M_Buck7, M_Buck8);
```

```
%% Visualization
```

```
SFD_FAIL = zeros(1,length(x));
```

```
BMD_FAIL = zeros(1,length(x));
```

```
[SFD_FAIL, BMD_FAIL] = ApplyPL(550, Pf, x, SFD_FAIL, locationA, locationB); %  
Construct SFD, BMD %These are at the failure load calculated above
```

```
[SFD_FAIL, BMD_FAIL] = ApplyPL(L, Pf, x, SFD_FAIL, locationA, locationB); %  
Construct SFD, BMD
```

```
VisualizePL(x, SFD_FAIL, BMD_FAIL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC,  
M_Buck1, M_Buck2, M_Buck3,M_Buck4,M_Buck5,M_Buck6, M_Buck7, M_Buck8, Pf);
```

```
%Our Design
```

```
%Train:
```

```
%diaphragms
```

```
a = zeros(1,length(x)); %Our design has 3 different diaphragm spacings
```

```
a(1:550) = 104;
```

```
a(551:1060) = 120;
```

```
a(1060:1250) = 53;
```

```
%% 2. Define cross-sections
```

```
% Cross Section A
```

```
dx = n/L;
```

```
xc = [1 dx*350 dx*738 dx*838 dx*L]; % x locations of changes in cross section
```

```
I = zeros(1, length(x));
```

```
yBar = zeros(1, length(x));  
Qglue = zeros(1, length(x));  
Qmax = zeros(1, length(x));
```

```
%Cross Section A
```

```
bVector = [100 1.27 1.27 10 10];  
hVector = [1.27 120 120 1.27 1.27];  
yVector = [120.635 60 60 119.365 119.365]; % distances between local centroid  
of subcomponents of sections from bottom  
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector,  
yVector); %This is called twice because the cross-section is used in two  
different separate spots  
[I, yBar] = SectionProperties( I, yBar, xc(3)+1, xc(4), bVector, hVector,  
yVector);  
cutoffAreaCent = [ 1.27*yBar(1), 1.27*yBar(1)];  
yCentCutoff = [ yBar(1)/2, yBar(1)/2];  
cutoffAreaGlue = [127];  
yGlue = [121.27-yBar(1)-0.635]; % centroid of the cutoff area  
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(1), xc(2), cutoffAreaCent,  
cutoffAreaGlue, yCentCutoff, yGlue); %This is called twice because the  
cross-section is used in two different separate spots  
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(3)+1, xc(4), cutoffAreaCent,  
cutoffAreaGlue, yCentCutoff, yGlue);
```

```
% Cross Section B
```

```
bVector = [100 1.27 1.27 10 10 10 10 70 70];  
hVector = [1.27 117.46 117.46 1.27 1.27 1.27 1.27 1.27 1.27];  
yVector = [120.635 61.27 61.27 119.365 119.365 3.175 3.175 1.905 0.635]; %  
distances between local centroid of subcomponents of sections from bottom  
[I, yBar] = SectionProperties( I, yBar, xc(2)+1, xc(3), bVector, hVector,  
yVector); %This is called twice because the cross-section is used in two  
different separate spots
```

```

[I, yBar] = SectionProperties( I, yBar, xc(4)+1, xc(5), bVector, hVector,
yVector);

cutoffAreaCent = [ 127 12.7 12.7 (120-yBar(900))*1.27 (120-yBar(900))*1.27];

yCentCutoff = [ 121.27-yBar(900)-0.635 120-yBar(900)-0.635 120-yBar(900)-0.635
120-yBar(900)/2 120-yBar(900)/2];

cutoffAreaGlue = [127];

yGlue = [121.27-yBar(1)-0.635]; % centroid of the cutoff area

[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(2)+1, xc(3), cutoffAreaCent,
cutoffAreaGlue, yCentCutoff, yGlue); %This is called twice because the
cross-section is used in two different separate spots

[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(4)+1, xc(5), cutoffAreaCent,
cutoffAreaGlue, yCentCutoff, yGlue);

```

```

%Other constants

b = zeros(1,length(x));

b(:) = 2*1.27;

bGlue = zeros(1,length(x));

bGlue(:) = 10;

t = zeros(1,length(x));

t(:) = 1.27;

height = zeros(1,length(x));

height(:) = 120;

```

```

% It was unnecessary to calculate the SFD and BMD again since it was already
% done for Design 0

```

```

%Find applied stresses

%Shear Stress demand

ShearStress = VStress(I, b, Qmax, SFD_TrainLoad);

ShearStressGlue = VStress(I, 20 + 2*1.27, Qglue, SFD_TrainLoad);

```

```
%Shear Stress Capacity
```

```
ShearBuckle = VBuck(I, t, b, height, E, mu, yBar, a, Qmax);
```

```
%Bending Stress
```

```
%Bending Stress Demand
```

```
BendingStressT = MStressT( I,yBar, height, BMD_TrainLoad);
```

```
BendingStressC = MStressC( I,yBar, height, BMD_TrainLoad);
```

```
%Shear Stress Capacity
```

```
%Cross Section A
```

```
BendingBuck1 = MBuck( I, t, 15, E, mu, 2);
```

```
BendingBuck2 = MBuck( I, t, 70-2*(1.27), E, mu, 1);
```

```
BendingBuck3 = MBuck( I, t, 39.859, E, mu, 3 );
```

```
BendingBuck4 = MBuck( I, t, 80.1411, E, mu, 3 );
```

```
%Cross Section B
```

```
BendingBuck5 = MBuck( I, t, 56.4858-2*1.27, E, mu, 3 );
```

```
BendingBuck6 = MBuck( I, t, 117.46-56.4858, E, mu, 3 );
```

```
BendingBuck7 = MBuck( I, 2*t, 70-2*1.27, E, mu, 1 );
```

```
BendingBuck8 = MBuck( I, t, 56.4858-2*1.27, E, mu, 3 );
```

```
FOSBridge = FOSCalc(ShearStress, ShearStressGlue, BendingStressT,  
BendingStressC, SigT, SigC, BendingBuck1, BendingBuck2, BendingBuck3,  
BendingBuck4, BendingBuck5, BendingBuck6, BendingBuck7, BendingBuck8, TauU,  
TauG, ShearBuckle);
```

```
%Visualization
```

```
V_Mat = Vfail(I, b, Qmax, TauU);
```

```
V_Buck = VfailBuck( I, t, b, height, E, mu, yBar, a, TauU, Qmax);
```

```

V_Glue = Vfail(I, bGlue, Qglue, TauU);
M_MatT = MfailMatT(I, yBar, height, SigT, BMD_TrainLoad);
M_MatC = MfailMatC(I, yBar, height, SigC, BMD_TrainLoad);
M_Buck1 = MfailBuck(I, t, 15, E, mu, BMD_TrainLoad, 2, yBar, height, SigC);
M_Buck2 = MfailBuck(I, t, 70-2*(1.27), E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck3 = MfailBuck(I, t, 39.859, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck4 = MfailBuck(I, t, 80.1411, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck5 = MfailBuck(I, t, 56.4858-2*1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck6 = MfailBuck(I, t, 117.46-56.4858, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);
M_Buck7 = MfailBuck(I, 2*t, 70-2*1.27, E, mu, BMD_TrainLoad, 1, yBar, height, SigC);
M_Buck8 = MfailBuck(I, t, 56.4858-2*1.27, E, mu, BMD_TrainLoad, 3, yBar, height, SigC);

%% Visualization
VisualizeTL(x, SFD_TrainLoad, BMD_TrainLoad, SFD_TrainLoad2, BMD_TrainLoad2, SFD_TrainLoad3, BMD_TrainLoad3, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2, M_Buck3, M_Buck4, M_Buck5, M_Buck6, M_Buck7, M_Buck8, FOSBridge);

```

```

%Point Load our Design:

```

```

Pf = FailLoad( SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2, M_Buck3, M_Buck4, M_Buck5, M_Buck6, M_Buck7, M_Buck8);

[SFD_FAIL, BMD_FAIL] = ApplyPL(550, Pf, x, SFD_FAIL, locationA, locationB); % Construct SFD, BMD

[SFD_FAIL, BMD_FAIL] = ApplyPL(L, Pf, x, SFD_FAIL, locationA, locationB); % Construct SFD, BMD

VisualizePL(x, SFD_FAIL, BMD_FAIL, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2, M_Buck3, M_Buck4, M_Buck5, M_Buck6, M_Buck7, M_Buck8, Pf);

```

```

%% Train failure Functions:

```

```

function [ ShearFail ] = VStress(I, b, Qcent, SFD )
%Calculates the Shear Stress at Applied load for every x value

    ShearFail = ((SFD .* Qcent) ./ (I .* b));
end

```

```

function [ ShearBuckle ] = VBuck( I, t, b, height, E, mu, y, a, Qcent )
% Calculates the Shear Buckle capacity at every x value

    %pi
    pi = 355/113; %Pi approximation

```

```

    for i = 1:length(I)
        ShearBuckle(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 +
(t(i)/a(i))^2);
    end

```

```

end

```

```

function [ BendingTensionStress ] = MStressT( I,y, height, BMD ) %Tension
% Calculates Tension Stress at every x value based on applied load
BendingTensionStress = zeros(1,length(I));

```

```

for i = 1 : length(I)
    ybot = y(i);
    ytop = height(i) - y(i);

```

```

        if BMD(i) >= 0 % If the moment is positive, the tension failure will be
at the bottom

            BendingTensionStress(i) = (BMD(i) * ybot) / I(i);

        elseif BMD < 0 % If the moment is negative, the tension failure will be
at the top

            BendingTensionStress(i) = (BMD(i) * ytop) / I(i);

        end
    end
end

```

```

end

```

```

function [ BendingCompressionStress ] = MStressC( I,y, height, BMD )
%Compression
% % Calculates Compression Stress at every x value based on applied load

    BendingCompressionStress = zeros(1,length(I));

```

```

for i = 1:length(I)

```

```

    ybot = y(i);
    ytop = height(i) - y(i);

```

```

        if BMD(i) >= 0 % If the moment is positive, the compression failure will
be at the top

            BendingCompressionStress(i) = (BMD(i) * ytop) / I(i);

        elseif BMD(i) > 0 % If the moment is negative, the compression failure
will be at the bottom

            BendingCompressionStress(i) = (BMD(i) * ybot) / I(i);

        end
    end
end

```

```
end
```

```
function [ MBuckStress ] = MBuck( I, t, bBuck, E, mu, caseNum )
```

```
% Calculates Bending Capacity based on plate buckling for every x value
```

```
    if caseNum == 1
```

```
        k = 4;
```

```
    end
```

```
    if caseNum == 2
```

```
        k = 0.425;
```

```
    end
```

```
    if caseNum == 3
```

```
        k = 6;
```

```
    end
```

```
    for i = 1:length(I)
```

```
        MBuckStress(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;
```

```
    end
```

```
end
```

```
function [ FOS ] = FOSCalc( ShearFail, GlueFail, BendingTensionStress,  
BendingCompressionStress, TensionCapacity, CompressionCapacity,  
CompressionBucklingCapacity1, CompressionBucklingCapacity2,  
CompressionBucklingCapacity3, CompressionBucklingCapacity4,  
CompressionBucklingCapacity5, CompressionBucklingCapacity6,  
CompressionBucklingCapacity7, CompressionBucklingCapacity8, TauCapacity,  
GlueCapacity, TauBuckling )
```

```
    %Finds FOS based on every capacity and every demand -> FOS =
```

```
    %Capacity/Demand
```

```

    ShearFOS = min(TauCapacity ./ abs(ShearFail));
    GlueFOS = min(GlueCapacity ./ abs(GlueFail));
    ShearBucklingFOS = min(TauBuckling ./ abs(ShearFail));
    TensionFOS = min(TensionCapacity ./ abs(BendingTensionStress));
    CompressionFOS = min(CompressionCapacity ./ abs(BendingCompressionStress));
    CompressionBucklingFOS1 = min(CompressionBucklingCapacity1 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS2 = min(CompressionBucklingCapacity2 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS3 = min(CompressionBucklingCapacity3 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS4 = min(CompressionBucklingCapacity4 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS5 = min(CompressionBucklingCapacity5 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS6 = min(CompressionBucklingCapacity6 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS7 = min(CompressionBucklingCapacity7 ./
abs(BendingCompressionStress) );
    CompressionBucklingFOS8 = min(CompressionBucklingCapacity8 ./
abs(BendingCompressionStress) );

```

```

    %Finds the minimum of the FOS

    FOSes = [ShearFOS GlueFOS ShearBucklingFOS TensionFOS CompressionFOS
CompressionBucklingFOS1 CompressionBucklingFOS2 CompressionBucklingFOS3
CompressionBucklingFOS4 CompressionBucklingFOS5 CompressionBucklingFOS6
CompressionBucklingFOS7 CompressionBucklingFOS8];

    FOS = min(FOSes);
end

```

%Cross Section Functions

```

function [I, yBar] = SectionProperties( I, yBar, xStart, xEnd, bVector,
hVector, yVector ) % Calculates important sectional properties. Including but
not limited to ybar, I, Q, etc.

```

```

% Input: Geometric Inputs. Format will depend on user

% Output: Sectional Properties at every value of x. Each property is a 1-D
array of length n

% finding global centroid
for i = xStart : xEnd
    sumAY = 0;
    sumA = 0;
    for j = 1:length(bVector)
        sumAY = sumAY + bVector(j) * hVector(j) * yVector(j);
        sumA = sumA + bVector(j) * hVector(j);
    end
    yBar(i) = sumAY/sumA;
    %finding I
    for j = 1:length(bVector)
        I(i) = I(i) + (bVector(j)*hVector(j)^3)/12 +
bVector(j)*hVector(j)*(yVector(j) - yBar(i))^2;
    end
end
end

function [Qmax, Qglue] = calculateQ( Qmax, Qglue, xStart, xEnd, cutoffAreaCent,
cutoffAreaGlue, yCentCutoff, yGlueCutoff)

for i = xStart : xEnd
    %finding Q at centroid
    for j = 1:length(cutoffAreaCent)
        Qmax(i) = Qmax(i) + cutoffAreaCent(j)*yCentCutoff(j);
    end
    %finding Q at glue
    for j = 1:length(cutoffAreaGlue)
        Qglue(i) = Qglue(i) + cutoffAreaGlue(j)*yGlueCutoff(j);
    end
end

```

```
        end
    end
end
```

```
function [] = VisualizeTL(x, SFD_PL, BMD_PL, SFD2, BMD2, SFD3, BMD3, V_Mat,
V_Glue, V_Buck, M_MatT, M_MatC, M_Buck1, M_Buck2, M_Buck3, M_Buck4, M_Buck5,
M_Buck6, M_Buck7, M_Buck8, FOS)
```

```
    %Plots all outputs of design process
```

```
    figure;
```

```
    axis = zeros(1,length(x));
```

```
    hold on;
```

```
    %shear force diagram
```

```
    subplot(2, 3, 1)
```

```
    plot(x, SFD_PL, "k")
```

```
    hold on
```

```
    plot(x, SFD2, "k")
```

```
    hold on
```

```
    plot(x, SFD3, "k")
```

```
    hold on
```

```
    plot(x,axis,"k")
```

```
    hold off
```

```
    title1 = strcat("SFD: FOS of Bridge = ", num2str(FOS));
```

```
    title(title1)
```

```
    %shear force vs mat shear fail
```

```
    subplot(2, 3, 2)
```

```
    plot(x, SFD_PL, "k")
```

```
    hold on
```

```
    plot(x, V_Mat, "r")
```

```
    hold on
```

```
plot(x, V_Glue, "g")
hold on
plot(x, -V_Mat, "r")
hold on
plot(x, -V_Glue, "g")
hold on
plot(x, SFD2, "k")
hold on
plot(x, SFD3, "k")
hold on
hold on
plot(x,axis,"k")
hold off

legend("Shear Force", "Matboard Shear Failure", "Glue Shear Failure", 'Location', 'northwest', 'NumColumns', 2)

title("SFD vs Material Shear Failure")

subplot(2, 3, 3)
plot(x, SFD_PL, "k")
hold on
plot(x, V_Buck, "r")
hold on
plot(x, -V_Buck, "r")
hold on
plot(x, SFD2, "k")
hold on
plot(x, SFD3, "k")
hold on
plot(x,axis,"k")
hold off
```

```

    legend("Shear Force", "Buckling Failure",
'Location','northwest','NumColumns',2)

    title("SFD vs Shear Buckling Failure")

    %BMD Diagrams
    subplot(2, 3, 4)
    plot(x, BMD_PL, "k")
    hold on
    plot(x, BMD2, "k")
    hold on
    plot(x, BMD3, "k")
    hold on
    plot(x,axis,"k")
    hold on
    title2 = strcat("BMD:");
    title(title2)
    set(gca, 'YDir', 'reverse')
    set(gca, 'YDir', 'reverse')
    hold off
    subplot(2, 3, 5)
    plot(x, BMD_PL, "k")
    hold on
    plot(x, M_MatT, "r")
    hold on
    plot(x, M_MatC, "b")
    hold on
    plot(x, BMD2, "k")
    hold on
    plot(x, BMD3, "k")
    hold on
    plot(x,axis,"k")

```

```
hold on

legend("Bending Moment", "Matboard Tension Failure", "Matboard Compression Failure", 'Location','northwest','NumColumns',2)

title("BMD vs Material Moment Failures")

set(gca, 'YDir', 'reverse')

hold off

subplot(2, 3, 6)

plot(x, BMD_PL, "k")

hold on

plot(x, M_Buck1, "-r")

hold on

plot(x, M_Buck2, 'Color', [0.74, 0.36, 0.04])

hold on

plot(x, M_Buck3, 'Color', [0.4, 0.75, 0.13])

hold on

plot(x, M_Buck4, "-r")

hold on

plot(x, M_Buck5, 'Color', [0.4, 0.75, 0.13])

hold on

plot(x, M_Buck6, 'Color', [0.4, 0.75, 0.13])

hold on

plot(x, M_Buck7, 'Color', [0.74, 0.36, 0.04])

hold on

plot(x, M_Buck8, 'Color', [0.4, 0.75, 0.13])

hold on

plot(x, BMD2, "k")

hold on

plot(x, BMD3, "k")

hold on

plot(x,axis,"k")
```

```

    hold on

    %legend("Bending Moment", "Mid Flange Buckling", "Side Flange Buckling", "Web
    Compression Buckling", 'Location','northwest','NumColumns',2)

    title("BMD vs Moment Buckling Failures")

    legend("Bending Moment", "Side Flange", "Mid Flange", "Web Flange",
    'Location','northwest','NumColumns',2)

    set(gca, 'YDir', 'reverse')

    hold off
end

```

%%Point Load Functions:

```

function [ SFD, BMD ] = ApplyPL( xP, P, x, SFD, locationA, locationB )
%(Location, Force, x coordinate, )

```

% Constructs SFD and BMD for every x value

% Can be iterated since SFD and BMD diagrams can be added together

%Calculate reaction forces

SupportB = ((xP * P) / (locationB)); %Using Sum of Moments = 0

SupportA = (P - SupportB); %Using sum of vertical force = 0

%Fill out SFD diagram as if by hand

SFDLocal = zeros(1, length(SFD));

if xP < locationB %Force is closer to the left side than support B

for i = 1:(xP-1) %SFD before new load

SFDLocal(i) = SupportA;

end

for i = xP:locationB %SFD after new load

SFDLocal(i) = (SupportA - P);

```

end

for i = 1:length(SFD) %add the new SFD with the old ones
    SFD(i) = SFD(i) + SFDLocal(i);
end

else %Force is farther from left side than support B
    for i = 1:(locationB-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = locationB:xP-1 %SFD after new load
        SFDLocal(i) = (SupportA + SupportB);
    end
end

```

```

for i = xP:length(SFD)
    SFDLocal(i) = 0;
end

```

```

for i = 1:length(SFD) %add the new SFD with the old ones
    SFD(i) = SFD(i) + SFDLocal(i);
end

end

%form BMD:
dx = x(2) - x(1);
BMD = cumsum((SFD * dx));

end

```

```

function [ V_fail ] = Vfail(I, b, Qcent, TauU )
% Calculates Shear Fail based on Tau Ultimate
    V_fail = TauU .* I .* b ./ Qcent;
end
function [ V_Buck ] = VfailBuck( I, t, b, height, E, mu, y, a, TauU, Qcent )
% Calculates Shear Fail based on Buckling

    %pi
    pi = 355/113;

```

```

    for i = 1:length(I)
        sigma(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 +
(t(i)/a(i))^2);
    end

    %Calculate buckle fail by using the above formula
    V_Buck = Vfail(I, b, Qcent, sigma);

```

```

end

```

```

function [ M_MatT ] = MfailMatT( I,y, height, SigT, BMD )
% Calculates Tension Fail
    for i = 1 : length(I)

```

```

        ybot = y;
        ytop = height(i) - y; %I need somesort of height vector

```

```

        if BMD(i) > 0 % If the moment is positive, the tension failure will be
at the bottom
            M_MatT(i) = (SigT * I(i)) / ybot(i);

```

```

        elseif BMD(i) < 0 % If the moment is negative, the tension failure will
be at the top
            M_MatT(i) = (-SigT * I(i)) / ytop(i);
        end
    end
end
function [ M_MatC ] = MfailMatC( I,y, height, SigC, BMD )
% Calculates Compression Fail

```

```

ybot = y;
ytop = height - y;
for i = 1 : length(I)
    if BMD(i) > 0 % If the moment is positive, the compression failure will
be at the top
        M_MatC(i) = (SigC * I(i)) / ytop(i);
    elseif BMD(i) < 0 % If the moment is negative, the compression failure
will be at the bottom
        M_MatC(i) = (-SigC * I(i)) / ybot(i);
    end
end
end
function [ M_Buck ] = MfailBuck( I, t, bBuck, E, mu, BMD, caseNum, y, height,
SigC )
% Calculates Bending Fail based on buckling

%Assign a k value based on the case
if caseNum == 1
    k = 4;
end
if caseNum == 2
    k = 0.425;

```

```

end

if caseNum == 3

    k = 6;

end

```

```

for i = 1:length(I)

    sigma(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;

end

```

```

%Calculate buckle fail using the above function

M_Buck = MfailMatC(I,y, height, min(sigma), BMD);

```

```

end

```

```

function [ Pf ] = FailLoad( SFD, BMD, V_Mat, V_Glue, V_Buck, M_MatT, M_MatC,
M_buck1, M_buck2, M_buck3, M_buck4, M_buck5, M_buck6, M_buck7, M_buck8 )

%Calculate the point load that causes the failure

```

```

%Flexural Stress

%Because P = 1, the BMD and SFD are simply the coefficients of P. By
%dividing them, we are finding P despite the fact that there is no actual P
%variable.

CompressionFailure = abs(M_MatC) ./ abs(BMD);

TensionFailure = abs(M_MatT) ./ abs(BMD);

BendingBucklingFailure1 = abs(M_buck1) ./ abs(BMD);
BendingBucklingFailure2 = abs(M_buck2) ./ abs(BMD);
BendingBucklingFailure3 = abs(M_buck3) ./ abs(BMD);
BendingBucklingFailure4 = abs(M_buck4) ./ abs(BMD);
BendingBucklingFailure5 = abs(M_buck5) ./ abs(BMD);
BendingBucklingFailure6 = abs(M_buck6) ./ abs(BMD);

```

```
BendingBucklingFailure7 = abs(M_buck7) ./ abs(BMD);  
BendingBucklingFailure8 = abs(M_buck8) ./ abs(BMD);
```

```
%Shear Stress
```

```
ShearFailureMat = V_Mat ./ abs(SFD);  
ShearFailureGlue = V_Glue ./ abs(SFD);  
ShearBucklingFailure = V_Buck ./ abs(SFD);
```

```
%Find the lowest Pf
```

```
Pforces = [min(CompressionFailure) min(TensionFailure)  
min(BendingBucklingFailure1) min(BendingBucklingFailure2)  
min(BendingBucklingFailure3) min(BendingBucklingFailure4)  
min(BendingBucklingFailure5) min(BendingBucklingFailure6)  
min(BendingBucklingFailure7) min(BendingBucklingFailure8) min(ShearFailureMat)  
min(ShearFailureGlue) min(ShearBucklingFailure)];  
Pf = min(Pforces);
```

```
end
```

```
function [] = VisualizePL(x, SFD_PL, BMD_PL, V_Mat, V_Glue, V_Buck, M_MatT,  
M_MatC, M_Buck1, M_Buck2, M_Buck3, M_Buck4, M_Buck5, M_Buck6, M_Buck7, M_Buck8,  
Pf)
```

```
axis = zeros(1,length(x));  
%Plots all outputs of design process  
figure;  
hold on;  
%shear force diagram  
subplot(2, 3, 1)  
plot(x, SFD_PL, "k")  
hold on
```

```

    plot(x,axis,"k")

    hold off

    title1 = strcat("SFD from P = ", int2str(floor(Pf)), "N Pfail = ",
num2str(Pf), " N");

    title(title1)

    %shear force vs mat shear fail

    subplot(2, 3, 2)

    plot(x, SFD_PL, "k")

    hold on

    plot(x, V_Mat, "r")

    hold on

    plot(x, V_Glue, "g")

    hold on

    plot(x, -V_Mat, "r")

    hold on

    plot(x, -V_Glue, "g")

    hold on

    hold on

    plot(x,axis,"k")

    legend("Shear Force", "Matboard Shear Failure", "Glue Shear
Failure",'Location','northwest','NumColumns',2)

    title("SFD vs Material Shear Failure")

    hold off

    subplot(2, 3, 3)

    plot(x, SFD_PL, "k")

    hold on

    plot(x, V_Buck, "r")

    hold on

    plot(x, -V_Buck, "r")

    hold on

```

```

hold on

plot(x,axis,"k")

legend("Shear Force", "Buckling Failure",
'Location','northwest','NumColumns',2)

title("SFD vs Shear Buckling Failure")

hold off

%BMD Diagrams

subplot(2, 3, 4)

plot(x, BMD_PL, "k")

hold on

hold on

plot(x,axis,"k")

title2 = strcat("BMD from P = ", int2str(floor(Pf)), "N Pfail = ",
int2str(Pf), " N");

title(title2)

set(gca, 'YDir', 'reverse')

set(gca, 'YDir', 'reverse')

hold off

subplot(2, 3, 5)

plot(x, BMD_PL, "k")

hold on

plot(x, M_MatT, "r")

hold on

plot(x, M_MatC, "b")

hold on

plot(x,axis,"k")

hold on

legend("Bending Moment", "Matboard Tension Failure", "Matboard Compression
Failure", 'Location','northwest','NumColumns',2)

title("BFD vs Material Moment Failures")

```

```

set(gca, 'YDir', 'reverse')
hold off

subplot(2, 3, 6)
plot(x, BMD_PL, "-k")
hold on
plot(x, M_Buck1, "-r")
hold on
plot(x, M_Buck2, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck3, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck4, "-r")
hold on
plot(x, M_Buck5, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck6, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x, M_Buck7, 'Color', [0.74, 0.36, 0.04])
hold on
plot(x, M_Buck8, 'Color', [0.4, 0.75, 0.13])
hold on
plot(x,axis,"k")
hold on

%legend("Bending Moment", "Mid Flange Buckling", "Side Flange Buckling", "Web
Compression Buckling", 'Location','northwest','NumColumns',2)

title("BMD vs Moment Buckling Failures")

legend("Bending Moment", "Side Flange", "Mid Flange", "Web Flange",
'Location','northwest','NumColumns',2)

set(gca, 'YDir', 'reverse')
hold off

```

end

```

%This is all explained in the other program
%% Intial Parameters for both designs and cases:
n = 1250;           % Number of locations to evaluate bridge failure
L = 1250;           % Length of bridge

x = linspace(1,L,n); % Define x coordinate

locationA = 1;
locationB = 1060;

E = 4000;

%%Design 0
%Cross Section
dx = n/L;
xc = [1 dx*L]; % x locations of changes in cross section
bVector = [100 1.27 1.27 10 10 80];
hVector = [1.27 72.46 72.46 1.27 1.27 1.27];
yVector = [74.365 37.5 37.5 73.1 73.1 0.635]; % distances between local centroid of sub

I = zeros(1, length(x));
yBar = zeros(1, length(x));
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector);

P = 200;

SFD_200 = zeros(1, length(x)); % Initialize SFD(x)
BMD_200 = zeros(1,length(x));

locationA = 1;
locationB = 1060;

[SFD_200, BMD_200] = ApplyPL(550, P, x, SFD_200, locationA, locationB); % Construct SFD
[SFD_200, BMD_200] = ApplyPL(L, P, x, SFD_200, locationA, locationB); % Construct SFD

%Find Deflection between supports
designZeroDeflection = deflectionCalc(x, BMD_200, I, E);
disp("Design 0")

```

Design 0

```
disp(designZeroDeflection)
```

1.3655

```
%%Our design
```

```

%This is all explained in the other program
%% 2. Define cross-sections
% Cross Section A
dx = n/L;
xc = [1 dx*350 dx*738 dx*838 dx*L]; % x locations of changes in cross section
I = zeros(1, length(x));

```

```

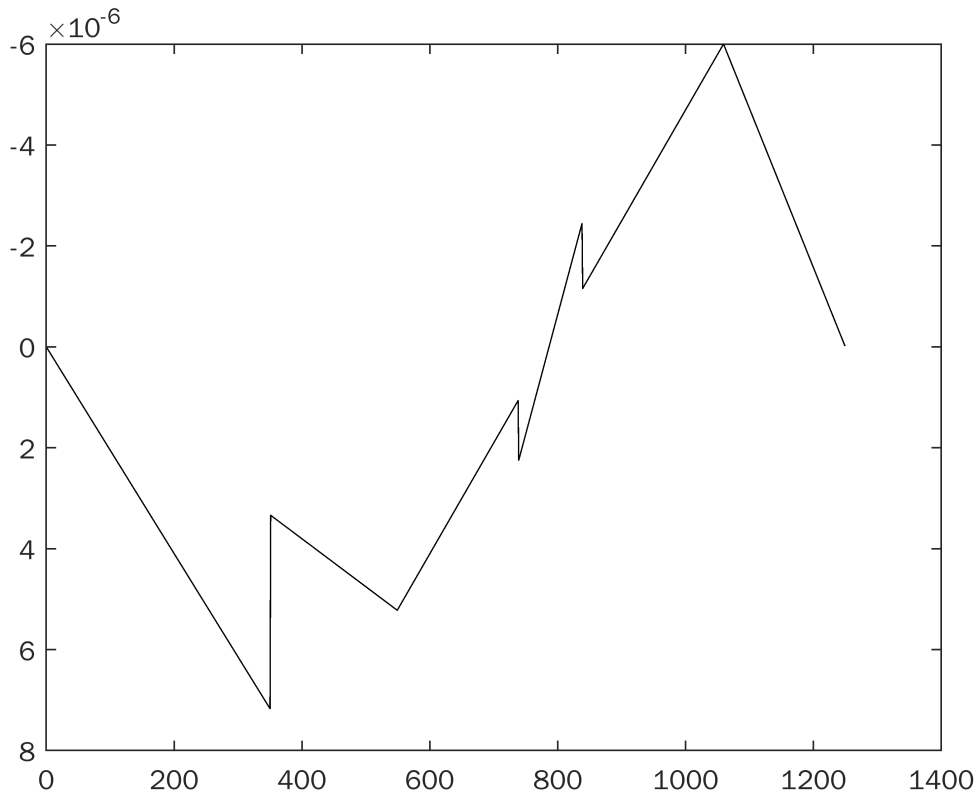
yBar = zeros(1, length(x));
Qglue = zeros(1, length(x));
Qmax = zeros(1, length(x));

%Cross Section A
bVector = [100 1.27 1.27 10 10];
hVector = [1.27 120 120 1.27 1.27];
yVector = [120.635 60 60 119.365 119.365]; % distances between local centroid of subcom
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector);
[I, yBar] = SectionProperties( I, yBar, xc(3)+1, xc(4), bVector, hVector, yVector);

% Cross Section B
bVector = [100 1.27 1.27 10 10 10 10 70 70];
hVector = [1.27 117.46 117.46 1.27 1.27 1.27 1.27 1.27 1.27];
yVector = [120.635 61.27 61.27 119.365 119.365 3.175 3.175 1.905 0.635]; % distances be
[I, yBar] = SectionProperties( I, yBar, xc(2)+1, xc(3), bVector, hVector, yVector);
[I, yBar] = SectionProperties( I, yBar, xc(4)+1, xc(5), bVector, hVector, yVector);

%Find Deflection
designZeroDeflection = deflectionCalc(x, BMD_200, I, E);

```



```
disp("Our Design")
```

Our Design

```
disp(designZeroDeflection)
```

0.4367

```

function [ SFD, BMD ] = ApplyPL( xP, P, x, SFD, locationA, locationB ) %(Location, Force)
% Constructs SFD and BMD from application of 1 Point Load. Assumes fixed location of supports
% Input: location and magnitude of point load. The previous SFD can be entered as input
% construct SFD of multiple point loads
% Output: SFD, BMD both 1-D arrays of length n

SupportB = ((xP * P) / (locationB)); %Using Sum of Moments = 0
SupportA = (P - SupportB); %Using sum of vertical force = 0

%find index of x value = xP
indexXP = length(SFD);
for i = 1:length(x)
    if indexXP == length(SFD)
        if x(i) >= xP
            indexXP = i;
        end
    end
end

SFDLocal = zeros(1, length(SFD));
if xP < locationB
    for i = 1:(xP-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = xP:locationB %SFD after new load
        SFDLocal(i) = (SupportA - P);
    end

    for i = 1:length(SFD) %add the new SFD with the old ones
        SFD(i) = SFD(i) + SFDLocal(i);
    end
else
    for i = 1:(locationB-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = locationB:xP-1 %SFD after new load
        SFDLocal(i) = (SupportA + SupportB);
    end

    for i = xP:length(SFD)
        SFDLocal(i) = 0;
    end

    for i = 1:length(SFD) %add the new SFD with the old ones
        SFD(i) = SFD(i) + SFDLocal(i);
    end
end
end

```

```

    %form BMD:
    dx = x(2) - x(1);
    BMD = cumsum((SFD * dx));
end

function [I, yBar] = SectionProperties( I, yBar, xStart, xEnd, bVector, hVector, yVector)
% Input: Geometric Inputs. Format will depend on user
% Output: Sectional Properties at every value of x. Each property is a 1-D array of

% finding global centroid
for i = xStart : xEnd
    sumAY = 0;
    sumA = 0;

    for j = 1:length(bVector)
        sumAY = sumAY + bVector(j) * hVector(j) * yVector(j);
        sumA = sumA + bVector(j) * hVector(j);
    end

    yBar(i) = sumAY/sumA;

%finding I
    for j = 1:length(bVector)
        I(i) = I(i) + (bVector(j)*hVector(j)^3)/12 + bVector(j)*hVector(j)*(yVector(j)-yBar(i))^2;
    end

end

end

function [Defls] = deflectionCalc(x, BMD, I, E )
%Get Curvature diagram
phi = (BMD)./( I * E);
subplot(1,1,1)
plot(x,phi,"k")
set(gca, 'YDir', 'reverse')

%Calculate distance of centroid to point of interest
dx = x(2) - x(1);
xBar1 = zeros(1,length(x));
xBar2 = zeros(1,length(x));

for i = 1:530
    xBar1(i) = x(530) - x(i);
end

for i = 1:1060
    xBar2(i) = x(1060) - x(i);
end

%Calculate distance from deflection to tangent line at support A
delta_BA = sum(phi .* dx .* xBar1);
delta_DA = sum(phi .* dx .* xBar2);

```

```
%Use similar triangles to find deflection in between supports  
Defls = (530 * abs(delta_DA))/1060 - abs(delta_BA);  
end
```

Function:

```
function [ ShearFail ] = VStress(I, b, Qcent, SFD )  
%Calculates the Shear Stress at Applied load for every x value  
  
    ShearFail = ((SFD .* Qcent) ./ (I .* b));  
end
```

Sample Call:

```
VStress(415.684*10^3, 2*1.27, 6248.5, 171.4465)
```

Code Result:

```
ans = 1.0146
```

Hand Calculation:

$$\begin{aligned}\tau &= \frac{VQ}{Ib} \\ &= \frac{(171.4465 \text{ N})(6248.5 \text{ mm}^3)}{(415.684 \times 10^3 \text{ mm}^4)(2 \times 1.27)} \\ &= 1.014629237 \text{ MPa} \\ &= 1.0146 \text{ MPa}\end{aligned}$$

Function:

```
function [ ShearBuckle ] = VBuck( I, t, b, height, E, mu, y, a, Qcent )  
% Calculates the Shear Buckle capacity at every x value  
  
%pi  
pi = 355/113; %Pi approximation  
  
for i = 1:length(I)  
    ShearBuckle(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 + (t(i)/a(i))^2);  
end  
  
end
```

Sample Call:

```
VBuck(415.684*10^3, 1.2700, 2.5400, 75, 4000, 0.2, 41.7018, 550, 6248.5)
```

Code Result:

```
ans = 5.0045
```

Hand Calculation:

$$\begin{aligned}\sigma &= \frac{5\pi^2 E}{12(1-\mu^2)} \left(\left(\frac{t}{h} \right)^2 + \left(\frac{t}{a} \right)^2 \right) \\ &= \frac{5\pi^2 (4000 \text{ MPa})}{12(1-0.2^2)} \left(\left(\frac{1.27 \text{ mm}}{75 \text{ mm}} \right)^2 + \left(\frac{1.27 \text{ mm}}{550 \text{ mm}} \right)^2 \right) \\ &= 5.004535044 \text{ MPa} \\ &= 5.0045 \text{ MPa}\end{aligned}$$

Function:

```
function [ BendingTensionStress ] = MStressT( I,y, height, BMD ) %Tension
% Calculates Tension Stress at every x value based on applied load

BendingTensionStress = zeros(1,length(I));

for i = 1 : length(I)
    ybot = y(i);
    ytop = height(i) - y(i);

    if BMD(i) >= 0 % If the moment is positive, the tension failure will be at the bottom
        BendingTensionStress(i) = (BMD(i) * ybot) / I(i);
    elseif BMD < 0 % If the moment is negative, the tension failure will be at the top
        BendingTensionStress(i) = (BMD(i) * ytop) / I(i);
    end
end

end
```

Sample Call:

```
MStressT(415.684*10^3, 41.7018, 75, 171.4465)|
```

Code Result:

```
ans = 0.0172
```

Hand Calculation:

$$\begin{aligned}\sigma &= \frac{M y}{I} \\ &= \frac{(171.4465 \text{ Nmm})(41.7018 \text{ mm})}{415.684 \times 10^3 \text{ mm}^4} \\ &= 0.01719967 \text{ MPa} \\ &= 0.0172 \text{ MPa}\end{aligned}$$

Function:

```
function [ BendingCompressionStress ] = MStressC( I,y, height, BMD ) %Compression
%% Calculates Compression Stress at every x value based on applied load

BendingCompressionStress = zeros(1,length(I));

for i = 1:length(I)

    ybot = y(i);
    ytop = height(i) - y(i);

    if BMD(i) >= 0 % If the moment is positive, the compression failure will be at the top
        BendingCompressionStress(i) = (BMD(i) * ytop) / I(i);
    elseif BMD(i) < 0 % If the moment is negative, the compression failure will be at the bottom
        BendingCompressionStress(i) = (BMD(i) * ybot) / I(i);
    end
end
```

Sample Call:

```
MStressC(415.684*10^3, 41.7018, 75, 171.4465)
```

Code Result:

```
ans = 0.0137
```

Hand Calculation:

$$\begin{aligned}\sigma &= \frac{M y}{I} \\ &= \frac{(171.4465 \text{ Nmm})(75 - 41.7018 \text{ mm})}{415.684 \times 10^3 \text{ mm}^4} \\ &= 0.013733653 \text{ MPa} \\ &= 0.0137 \text{ MPa}\end{aligned}$$

Function:

```
function [ MBuckStress ] = MBuck( I, t, bBuck, E, mu, caseNum )
% Calculates Bending Capacity based on plate buckling for every x value

    if caseNum == 1
        k = 4;
    end
    if caseNum == 2
        k = 0.425;
    end
    if caseNum == 3
        k = 6;
    end

    for i = 1:length(I)
        MBuckStress(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;
    end

end
```

Sample Call:

```
MBuck(415.684*10^3, 1.2700, (100-80)/2, 4000, 0.2, 1)
```

Code Result:

```
ans = 221.0928
```

Hand Calculation:

$$\begin{aligned}\sigma &= \frac{4\pi^2 E}{12(1-\mu^2)} \left(\frac{t}{b} \right)^2 \\ &= \frac{4\pi^2 (4000 \text{ MPa})}{12(1-0.2^2)} \left(\frac{1.27}{\frac{(100-80)}{2}} \right)^2 \\ &= 221.0928464 \text{ MPa} \\ &= 221.0928 \text{ MPa}\end{aligned}$$

Function:

```
%Cross Section Functions
function [I, yBar] = SectionProperties( I, yBar, xStart, xEnd, bVector, hVector, yVector ) % Calculates in
% Input: Geometric Inputs. Format will depend on user
% Output: Sectional Properties at every value of x. Each property is a 1-D array of length n

% finding global centroid
for i = xStart : xEnd
    sumAY = 0;
    sumA = 0;

    for j = 1:length(bVector)
        sumAY = sumAY + bVector(j) * hVector(j) * yVector(j);
        sumA = sumA + bVector(j) * hVector(j);
    end

    yBar(i) = sumAY/sumA;

    %finding I
    for j = 1:length(bVector)
        I(i) = I(i) + (bVector(j)*hVector(j)^3)/12 + bVector(j)*hVector(j)*(yVector(j) - yBar(i))^2;
    end

end

end
```

Sample Call:

```
%Cross Section
%% 2. Define cross-sections
% Design 0
dx = n/L; %length between each x coordinate
xc = [1 dx*L]; % x locations of changes in cross section
bVector = [100 1.27 1.27 10 10 80]; %Base of rectangle segments
hVector = [1.27 72.46 72.46 1.27 1.27 1.27]; %height of rectangle segments
yVector = [74.365 37.5 37.5 73.1 73.1 0.635]; % distances between local centroid of subcomponents of sections from bottom

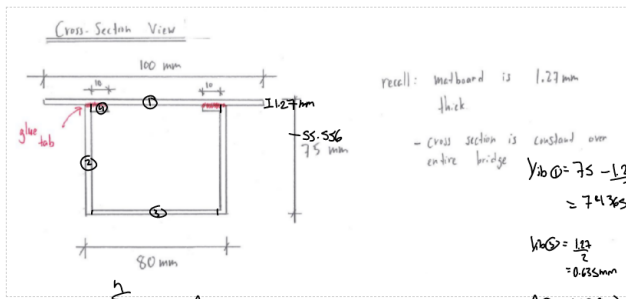
I = zeros(1, length(x)); %Initialize I values
yBar = zeros(1, length(x)); %Initialize Y values
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector); %Calculate I and Y values at every instance
```

Result:

```
I = 1x1250
105 x
4.1569

yBar = 1x1250
41.7018
```

Hand Calculation:



$$\bar{y} = \frac{\sum_{i=1}^n y_i \cdot A_i}{\sum_{i=1}^n A_i}$$

$$\sum_{i=1}^n A_i$$

17:

$$I = \sum_{i=1}^n (I_0 + A_i d_i^2)$$

$$= (I_0 + A_0 d_1^2) + 2(I_0 + A_0 d_2^2) + (I_0 + A_0 d_3^2) + 2(I_0 + A_0 d_4^2)$$

$$= 1555001.8 + 2(14609.28) + 165921.2 + 2(15972.8)$$

$$I = 415.684 \times 10^3 \text{ mm}^4$$

$$AC = (75 - \frac{1.27}{2}) - 41.702$$
$$= 32.663$$

$$dE = 41.722 - \left(\frac{75 - 1.27}{2} \right) = 4.837$$

$$d) ③ = 41.702 - \frac{1.27}{2} = 41.067$$

$$d_{(4)} = (75 - 1.27 - \frac{1.27}{2}) - 41.702$$

$$= 31.393$$

$$I_0 = \frac{100(1.27)^3}{12} = 17.069$$

$$I_{\text{Q}} = \frac{1.27(75-1.27)^3}{12} = 42418.5$$

$$I_0 = \frac{(86 - 2(1.77))(1.77)^3}{12} = 13.222$$

$$I_{\text{B}} = \frac{10(1.27)^2}{12} = 1.769 \text{ A}$$

Function:

```
function [Qmax, Qglue] = calculateQ( Qmax, Qglue, xStart, xEnd, cutoffAreaCent, cutoffAreaGlue, yCentCutoff, yGlueCutoff)

    for i = xStart : xEnd
        %finding Q at centroid
        for j = 1:length(cutoffAreaCent)
            Qmax(i) = Qmax(i) + cutoffAreaCent(j)*yCentCutoff(j);
        end

        %finding Q at glue
        for j = 1:length(cutoffAreaGlue)
            Qglue(i) = Qglue(i) + cutoffAreaGlue(j)*yGlueCutoff(j);
        end
    end

end
```

Sample Call:

```
%Cross Section
%% 2. Define cross-sections
% Design 0
dx = n/L; %length between each x coordinate
xc = [1 dx*L]; % x locations of changes in cross section
bVector = [100 1.27 1.27 10 10 80]; %Base of rectangle segments
hVector = [1.27 72.46 72.46 1.27 1.27 1.27]; %height of rectangle segments
yVector = [74.365 37.5 37.5 73.1 73.1 0.635]; % distances between local centroid of subcomponents of sections from bottom

I = zeros(1, length(x)); %Initialize I values
yBar = zeros(1, length(x)); %Initialize Y values
[I, yBar] = SectionProperties( I, yBar, xc(1), xc(2), bVector, hVector, yVector); %Calculate I and Y values at every instance

cutoffAreaCent = [ 77.46*1.27, 1.27*yBar(1), 1.27*yBar(1)]; %Calculate area from centroid for Q calc
yCentCutoff = [ yBar(1)-0.635, yBar(1)/2, yBar(1)/2]; %Calculate y values from centroid for Q calc

cutoffAreaGlue = [127];
yGlue = [75-yBar(1)-0.635]; % centroid of the cutoff area

Qglue = zeros(1, length(x)); %initialize Q at centriod and Q at glue
Qmax = zeros(1, length(x));
[Qmax, Qglue] = calculateQ( Qmax, Qglue, xc(1), xc(2), cutoffAreaCent, cutoffAreaGlue, yCentCutoff, yGlue); %Calculate Q at centroid and Q at glue
```

Code Result:

```
Qmax = 1x1250
      103 x
      6.2485
```

```
Qglue = 1x1250
      103 x
      4.1482
```

Hand Calculation:

$$Q_{max} = \sum A_i d_i \quad @ \bar{y}$$

$$= 2(A_1 d_1) + A_2 d_2$$

$$= 2(41.702)(1.27)(41.702/2) + 77.46(1.27)(41.702 - \frac{1.27}{2})$$

$$= 6248.5 \text{ mm}^3$$

$$Q_{max} = 6.25 \times 10^3 \text{ mm}^3$$

$$Q_{glue} = \sum A_i d_i$$

$$= 100(1.27)(76 - 41.702 - \frac{1.27}{2})$$

$$= 4148.2 \text{ mm}^3$$

$$Q_{glue} = 4.15 \times 10^3 \text{ mm}^3$$



d associated w/ max = 0
 d associated w/ glue = 32.7 mm



Function:

```
function [ SFD, BMD ] = ApplyPL( xP, P, x, SFD, locationA, locationB ) %(Location, Force, x coordinate, )
% Constructs SFD and BMD for every x value
% Can be iterated since SFD and BMD diagrams can be added together

%Calculate reaction forces
SupportB = ((xP * P) / (locationB)); %Using Sum of Moments = 0
SupportA = (P - SupportB); %Using sum of vertical force = 0

%Fill out SFD diagram as if by hand
SFDLocal = zeros(1, length(SFD));
if xP < locationB %Force is closer to the left side than support B
    for i = 1:(xP-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = xP:locationB %SFD after new load
        SFDLocal(i) = (SupportA - P);
    end

    for i = 1:length(SFD) %add the new SFD with the old ones
        SFD(i) = SFD(i) + SFDLocal(i);
    end
else %Force is farther from left side than support B
    for i = 1:(locationB-1) %SFD before new load
        SFDLocal(i) = SupportA;
    end

    for i = locationB:xP-1 %SFD after new load
        SFDLocal(i) = (SupportA + SupportB);
    end

    for i = xP:length(SFD)
        SFDLocal(i) = 0;
    end

    for i = 1:length(SFD) %add the new SFD with the old ones
        SFD(i) = SFD(i) + SFDLocal(i);
    end
end

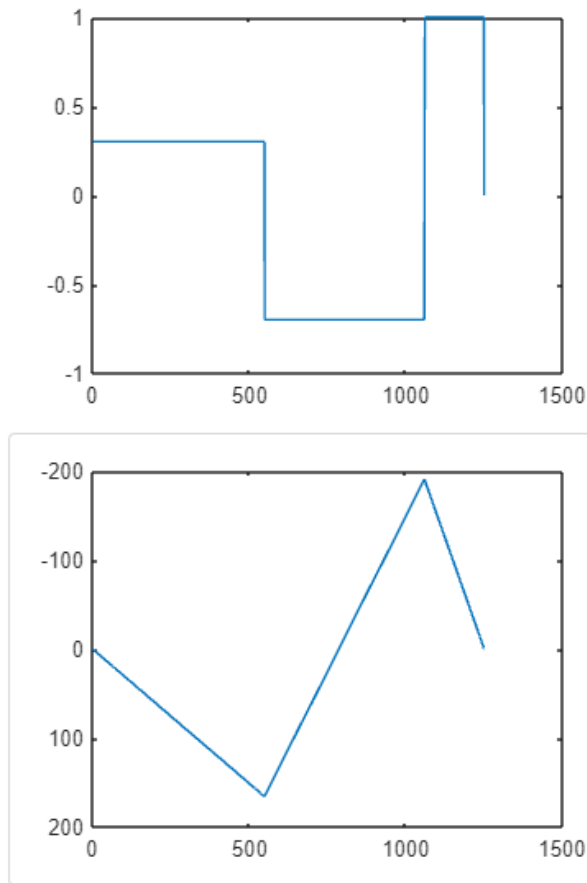
%form BMD:
dx = x(2) - x(1);
BMD = cumsum((SFD * dx));
end
```

Sample Call:

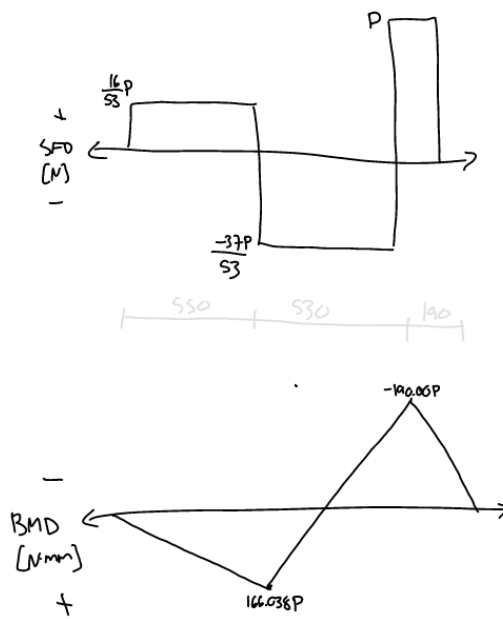
```
%Design 0 Point Loads
SFD_PL = zeros(1, n);      % Initialize SFD(x)
BMD_PL = zeros(1, n);
P = 1; %P = 1 * whatever value, so I can set p to 1

%% 1. Point Loading Analysis (SFD, BMD) Two loads for two points
[SFD_PL, BMD_PL] = ApplyPL(550, P, x, SFD_PL, locationA, locationB); % Construct SFD, BMD
[SFD_PL, BMD_PL] = ApplyPL(L, P, x, SFD_PL, locationA, locationB); % Construct SFD, BMD
```

Code Result:



Hand Calculation:



Function:

```
function [ V_fail ] = Vfail(I, b, Qcent, TauU )  
% Calculates Shear Fail based on Tau Ultimate  
  
    V_fail = TauU .* I .* b ./ Qcent;  
end
```

Sample Call:

```
Vfail(415.684*10^3, (100-80)/2, 6248.5, 4)
```

Code Result:

```
ans = 2.6610e+03
```

Hand Calculation:

$$V = \frac{(\tau)(I)(b)}{(Q)}$$
$$= \frac{(4 \text{ MPa})(415.684 \times 10^3 \text{ mm}^4)\left(\frac{100-80}{2} \text{ mm}\right)}{6248.5 \text{ mm}^3}$$
$$= 2661.016244 \text{ N}$$

Function:

```
function [ V_Buck ] = VfailBuck( I, t, b, height, E, mu, y, a, TauU, Qcent )
% Calculates Shear Fail based on Buckling

%pi
pi = 355/113;

for i = 1:length(I)
    sigma(i) = ((5*E*pi^2)/(12*(1-mu^2))) * ((t(i)/height(i))^2 + (t(i)/a(i))^2);
end

%Calculate buckle fail by using the above formula
V_Buck = Vfail(I, b, Qcent, sigma);

end
```

Sample Call:

```
VfailBuck( 415.684*10^3, 1.2700, (100-80)/2, 75, 4000, 0.2, 41.7018, 550, 4, 6248.5)
```

Code Result:

```
ans = 3.3293e+03
```

Hand Calculation:

$$\begin{aligned}\sigma &= \frac{5\pi^2 E}{12(1-\mu^2)} \left(\left(\frac{t}{h} \right)^2 + \left(\frac{t}{a} \right)^2 \right) \\ &= \frac{5\pi^2 (4000 \text{ MPa})}{12(1-0.2^2)} \left(\left(\frac{1.27 \text{ mm}}{75 \text{ mm}} \right)^2 + \left(\frac{1.27 \text{ mm}}{550 \text{ mm}} \right)^2 \right) \\ &= 5.004535044 \text{ MPa} \\ &= 5.0045 \text{ MPa}\end{aligned}$$

$$\begin{aligned}V &= \frac{(\tau)(I)(b)}{Q} \\ &= \frac{(5.0045 \text{ MPa})(415.684 \times 10^3 \text{ mm}^4) \left(\frac{100-80}{2} \text{ mm} \right)}{6248.5 \text{ mm}^3} \\ &= 3,329.263948 \text{ N}\end{aligned}$$

Function:

```
function [ M_MatT ] = MfailMatT( I,y, height, SigT, BMD )
% Calculates Tension Fail

for i = 1 : length(I)

    ybot = y;
    ytop = height(i) - y; %I need somesort of height vector

    if BMD(i) > 0 % If the moment is positive, the tension failure will be at the bottom
        M_MatT(i) = (SigT * I(i)) / ybot(i);
    elseif BMD(i) < 0 % If the moment is negative, the tension failure will be at the top
        M_MatT(i) = (-SigT * I(i)) / ytop(i);
    end
end
end
```

Sample Call:

```
MfailMatT(415.684*10^3, 41.7018, 75, 4,1)
```

Code Result:

```
ans = 2.9904e+05
```

Hand Calculation:

Zone A ⊕ M: 

$$M_{fail} = \frac{30 \text{ MPa} (415.684 \times 10^3 \text{ mm}^4)}{41.7018 \text{ mm}}$$
$$= 299046.1 \text{ Nmm}$$

$$M_{failT} = 299 \times 10^3 \text{ Nmm}$$

Function:

```
function [ M_MatC ] = MfailMatC( I,y, height, SigC, BMD )
% Calculates Compression Fail

ybot = y;
ytop = height - y;

for i = 1 : length(I)
    if BMD(i) > 0 % If the moment is positive, the compression failure will be at the top
        M_MatC(i) = (SigC * I(i)) / ytop(i);
    elseif BMD(i) < 0 % If the moment is negative, the compression failure will be at the bottom
        M_MatC(i) = (-SigC * I(i)) / ybot(i);
    end
end
end
```

Sample Call:

```
MfailMatC(415.684*10^3, 41.7018, 75, 6, -1)
```

Code Result:

```
ans = -5.9808e+04
```

Hand Calculation:

Zone B $\ominus M$ 

$$y_{bot} = \bar{y} = 41.7018 \text{ mm}$$
$$M_{fail} = \frac{\sigma_{fail} I}{y_{bot}}$$
$$= \frac{6 \text{ MPa} (415.684 \times 10^3 \text{ mm}^4)}{41.7018 \text{ mm}}$$
$$= 59809.2 \text{ Nmm}$$

$M_{failC} = 59.8 \times 10^3 \text{ Nmm}$

Function:

```
function [ M_Buck ] = MfailBuck( I, t, bBuck, E, mu, BMD, caseNum, y, height, SigC )
% Calculates Bending Fail based on buckling

%Assign a k value based on the case
if caseNum == 1
    k = 4;
end
if caseNum == 2
    k = 0.425;
end
if caseNum == 3
    k = 6;
end

for i = 1:length(I)
    sigma(i) = ((k*E*pi^2)/(12*(1-mu^2))) * (t(i)/bBuck)^2;
end

%Calculate buckle fail using the above function
M_Buck = MfailMatC(I,y, height, min(sigma), BMD);

end
```

Sample Call:

```
MfailBuck(415684, 1.27, (100-80)/2, 4000, 0.2, -1, 2, 33.299, 6)
```

Code Result:

```
ans = -2.9325e+05
```

Hand result:

Plate $\boxed{1}$, $\boxed{4}$: case 2 ($k=0.425$)

$$\begin{aligned}\sigma_{crit} &= \frac{4\pi^2 E}{12(1-\mu^2)} \left(\frac{t}{b}\right)^2 \\ &= \frac{0.425\pi^2 4000 \text{ MPa}}{12(1-0.2^2)} \left(\frac{1.27 \text{ mm}}{(100-80)/2}\right)^2 \\ &= 23.49 \text{ MPa}\end{aligned}$$

$$\begin{aligned}M_{fail} &= \frac{\sigma I}{y} \\ &= \frac{23.49 \text{ MPa} (415684 \text{ mm}^4)}{33.299 \text{ mm}}\end{aligned}$$

$$= 293234.5 \text{ Nmm}$$

$$M_{fail,1,3} = 293 \times 10^3 \text{ Nmm}$$